

tivities, as well as non-cortical artifact domains; see the reference for a detailed overview of ICA in this domain.

The lower part of Figure 14.41 shows a selection of ICA components. The colored images represent the estimated unmixing coefficient vectors $\hat{a}_j$ as heatmap images superimposed on the scalp, indicating the location of activity. The corresponding time-courses show the activity of the learned ICA components.

For example, the subject blinked after each performance feedback signal (colored vertical lines), which accounts for the location and artifact signal in IC1 and IC3. IC12 is an artifact associated with the cardiac pulse. IC4 and IC7 account for frontal theta-band activities, and appear after a stretch of correct performance. See Onton and Makeig (2006) for a more detailed discussion of this example, and the use of ICA in EEG modeling.

14.7.3 Exploratory Projection Pursuit

Friedman and Tukey (1974) proposed *exploratory projection pursuit*, a graphical exploration technique for visualizing high-dimensional data. Their view was that most low (one- or two-dimensional) projections of high-dimensional data look Gaussian. Interesting structure, such as clusters or long tails, would be revealed by non-Gaussian projections. They proposed a number of *projection indices* for optimization, each focusing on a different departure from Gaussianity. Since their initial proposal, a variety of improvements have been suggested (Huber, 1985; Friedman, 1987), and a variety of indices, including entropy, are implemented in the interactive graphics package Xgobi (Swayne et al., 1991, now called GGobi). These projection indices are exactly of the same form as $J(Y_j)$ above, where $Y_j = a_j^T X$, a normalized linear combination of the components of X . In fact, some of the approximations and substitutions for cross-entropy coincide with indices proposed for projection pursuit. Typically with projection pursuit, the directions a_j are not constrained to be orthogonal. Friedman (1987) transforms the data to look Gaussian in the chosen projection, and then searches for subsequent directions. Despite their different origins, ICA and exploratory projection pursuit are quite similar, at least in the representation described here.

14.7.4 A Direct Approach to ICA



Independent components have by definition a joint product density

$$f_S(s) = \prod_{j=1}^p f_j(s_j), \quad (14.88)$$

so here we present an approach that estimates this density directly using generalized additive models (Section 9.1). Full details can be found in

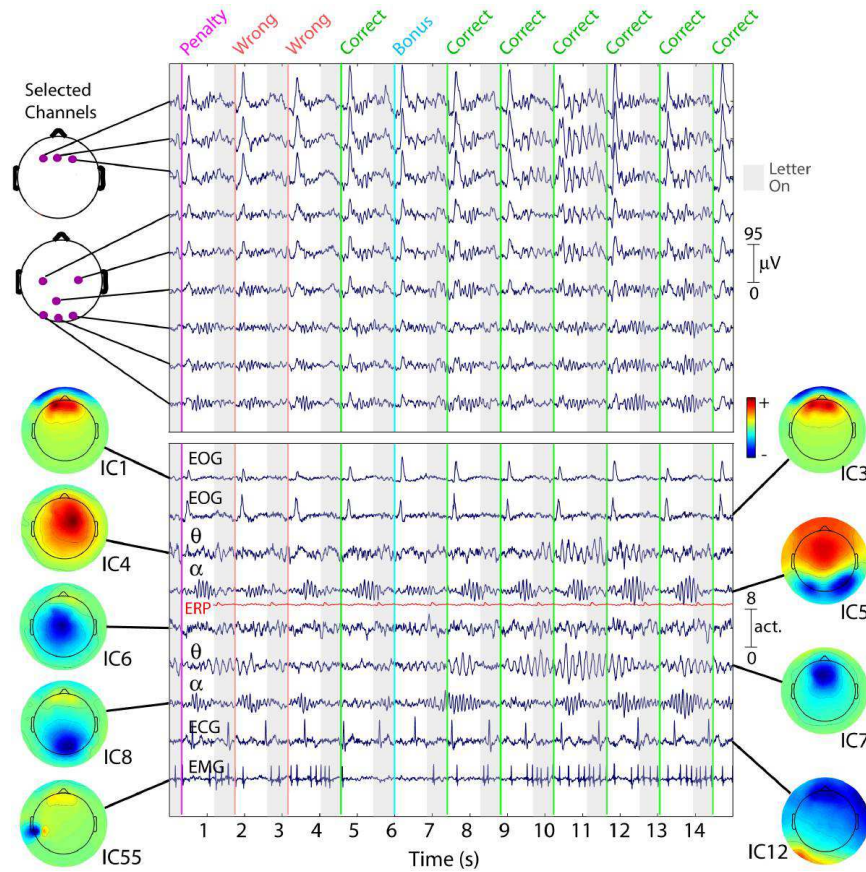


FIGURE 14.41. Fifteen seconds of EEG data (of 1917 seconds) at nine (of 100) scalp channels (top panel), as well as nine ICA components (lower panel). While nearby electrodes record nearly identical mixtures of brain and non-brain activity, ICA components are temporally distinct. The colored scalps represent the ICA unmixing coefficients $\hat{a}_j$ as a heatmap, showing brain or scalp location of the source.

Hastie and Tibshirani (2003), and the method is implemented in the R package **ProDenICA**, available from CRAN.

In the spirit of representing departures from Gaussianity, we represent each f_j as

$$f_j(s_j) = \phi(s_j)e^{g_j(s_j)}, \quad (14.89)$$

a *tilted* Gaussian density. Here ϕ is the standard Gaussian density, and g_j satisfies the normalization conditions required of a density. Assuming as before that X is pre-whitened, the log-likelihood for the observed data $X = \mathbf{A}S$ is

$$\ell(\mathbf{A}, \{g_j\}_1^p; \mathbf{X}) = \sum_{i=1}^N \sum_{j=1}^p [\log \phi_j(a_j^T x_i) + g_j(a_j^T x_i)], \quad (14.90)$$

which we wish to maximize subject to the constraints that $\mathbf{A}$ is orthogonal and that the g_j result in densities in (14.89). Without imposing any further restrictions on g_j , the model (14.90) is over-parametrized, so we instead maximize a regularized version

$$\sum_{j=1}^p \left[\frac{1}{N} \sum_{i=1}^N [\log \phi(a_j^T x_i) + g_j(a_j^T x_i)] - \int \phi(t)e^{g_j(t)} dt - \lambda_j \int \{g_j'''(t)\}^2(t) dt \right]. \quad (14.91)$$

We have subtracted two penalty terms (for each j) in (14.91), inspired by Silverman (1986, Section 5.4.4):

- The first enforces the density constraint $\int \phi(t)e^{\hat{g}_j(t)} dt = 1$ on any solution $\hat{g}_j$.
- The second is a roughness penalty, which guarantees that the solution $\hat{g}_j$ is a quartic-spline with knots at the observed values of $s_{ij} = a_j^T x_i$.

It can further be shown that the solution densities $\hat{f}_j = \phi e^{\hat{g}_j}$ each have mean zero and variance one (Exercise 14.18). As we increase λ_j , these solutions approach the standard Gaussian ϕ .

Algorithm 14.3 *Product Density ICA Algorithm: ProDenICA*

1. Initialize $\mathbf{A}$ (random Gaussian matrix followed by orthogonalization).
 2. Alternate until convergence of $\mathbf{A}$:
 - (a) Given $\mathbf{A}$, optimize (14.91) w.r.t. g_j (separately for each j).
 - (b) Given g_j , $j = 1, \dots, p$, perform one step of a fixed point algorithm towards finding the optimal $\mathbf{A}$.
-

We fit the functions g_j and directions a_j by optimizing (14.91) in an alternating fashion, as described in Algorithm 14.3.

Step 2(a) amounts to a semi-parametric density estimation, which can be solved using a novel application of generalized additive models. For convenience we extract one of the p separate problems,

$$\frac{1}{N} \sum_{i=1}^N [\log \phi(s_i) + g(s_i)] - \int \phi(t) e^{g(t)} dt - \lambda \int \{g'''(t)\}^2(t) dt. \quad (14.92)$$

Although the second integral in (14.92) leads to a smoothing spline, the first integral is problematic, and requires an approximation. We construct a fine grid of L values s_ℓ^* in increments Δ covering the observed values s_i , and count the number of s_i in the resulting bins:

$$y_\ell^* = \frac{\#s_i \in (s_\ell^* - \Delta/2, s_\ell^* + \Delta/2)}{N}. \quad (14.93)$$

Typically we pick L to be 1000, which is more than adequate. We can then approximate (14.92) by

$$\sum_{\ell=1}^L \left\{ y_\ell^* [\log(\phi(s_\ell^*)) + g(s_\ell^*)] - \Delta \phi(s_\ell^*) e^{g(s_\ell^*)} \right\} - \lambda \int g'''^2(s) ds. \quad (14.94)$$

This last expression can be seen to be proportional to a penalized Poisson log-likelihood with response y_ℓ^*/Δ and penalty parameter λ/Δ , and mean $\mu(s) = \phi(s) e^{g(s)}$. This is a *generalized additive spline model* (Hastie and Tibshirani, 1990; Efron and Tibshirani, 1996), with an *offset* term $\log \phi(s)$, and can be fit using a Newton algorithm in $O(L)$ operations. Although a quartic spline is called for, we find in practice that a cubic spline is adequate. We have p tuning parameters λ_j to set; in practice we make them all the same, and specify the amount of smoothing via the effective degrees-of-freedom $\text{df}(\lambda)$. Our software uses 5df as a default value.

Step 2(b) in Algorithm 14.3 requires optimizing (14.91) with respect to $\mathbf{A}$, holding the $\hat{g}_j$ fixed. Only the first terms in the sum involve $\mathbf{A}$, and since $\mathbf{A}$ is orthogonal, the collection of terms involving ϕ do not depend on $\mathbf{A}$ (Exercise 14.19). Hence we need to maximize

$$\begin{aligned} C(\mathbf{A}) &= \frac{1}{N} \sum_{j=1}^p \sum_{i=1}^N \hat{g}_j(a_j^T x_i) \\ &= \sum_{j=1}^p C_j(a_j) \end{aligned} \quad (14.95)$$

$C(\mathbf{A})$ is a log-likelihood ratio between the fitted density and a Gaussian, and can be seen as an estimate of negentropy (14.86), with each $\hat{g}_j$ a contrast function as in (14.87). The fixed point update in step 2(b) is a modified Newton step (Exercise 14.20)

1. For each j update

$$a_j \leftarrow E \{ X \hat{g}'_j(a_j^T X) - E[\hat{g}''_j(a_j^T X)] a_j \}, \quad (14.96)$$

where E represents expectation w.r.t the sample x_i . Since $\hat{g}_j$ is a fitted quartic (or cubic) spline, the first and second derivatives are readily available.

2. Orthogonalize $\mathbf{A}$ using the symmetric square-root transformation $(\mathbf{A}\mathbf{A}^T)^{-\frac{1}{2}}\mathbf{A}$. If $\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T$ is the SVD of $\mathbf{A}$, it is easy to show that this leads to the update $\mathbf{A} \leftarrow \mathbf{U}\mathbf{V}^T$.

Our **ProDenICA** algorithm works as well as **FastICA** on the artificial time series data of Figure 14.37, the mixture of uniforms data of Figure 14.38, and the digit data in Figure 14.39.

Example: Simulations

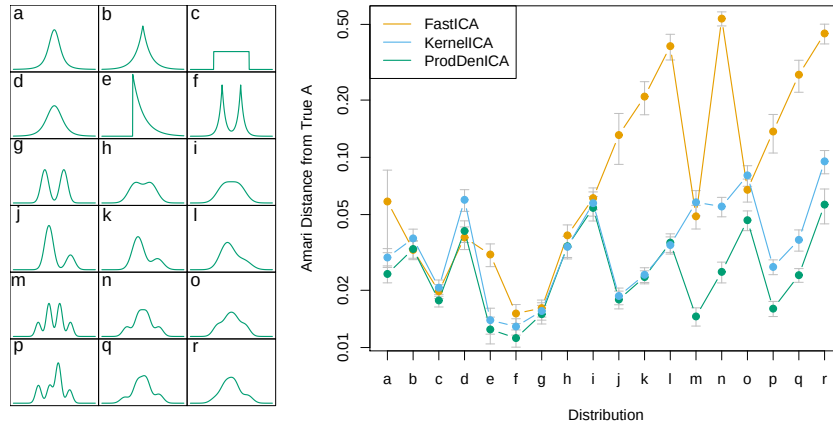


FIGURE 14.42. The left panel shows 18 distributions used for comparisons. These include the “ t ”, uniform, exponential, mixtures of exponentials, symmetric and asymmetric Gaussian mixtures. The right panel shows (on the log scale) the average Amari metric for each method and each distribution, based on 30 simulations in $\mathbb{R}^2$ for each distribution.

Figure 14.42 shows the results of a simulation comparing **ProDenICA** to **FastICA**, and another semi-parametric competitor **KernelICA** (Bach and Jordan, 2002). The left panel shows the 18 distributions used as a basis of comparison. For each distribution, we generated a pair of independent components ($N = 1024$), and a random mixing matrix in $\mathbb{R}^2$ with condition number between 1 and 2. We used our R implementations of **FastICA**, using the negentropy criterion (14.87), and **ProDenICA**. For **KernelICA** we used

the authors MATLAB code.¹² Since the search criteria are nonconvex, we used five random starts for each method. Each of the algorithms delivers an orthogonal mixing matrix $\mathbf{A}$ (the data were *pre-whitened*), which is available for comparison with the generating orthogonalized mixing matrix $\mathbf{A}_0$. We used the Amari metric (Bach and Jordan, 2002) as a measure of the closeness of the two frames:

$$d(\mathbf{A}_0, \mathbf{A}) = \frac{1}{2p} \sum_{i=1}^p \left(\frac{\sum_{j=1}^p |r_{ij}|}{\max_j |r_{ij}|} - 1 \right) + \frac{1}{2p} \sum_{j=1}^p \left(\frac{\sum_{i=1}^p |r_{ij}|}{\max_i |r_{ij}|} - 1 \right), \quad (14.97)$$

where $r_{ij} = (\mathbf{A}_0 \mathbf{A}^{-1})_{ij}$. The right panel in Figure 14.42 compares the averages (on the log scale) of the Amari metric between the truth and the estimated mixing matrices. *ProDenICA* is competitive with *FastICA* and *KernelICA* in all situations, and dominates most of the mixture simulations.

14.8 Multidimensional Scaling

Both self-organizing maps and principal curves and surfaces map data points in $\mathbb{R}^p$ to a lower-dimensional manifold. Multidimensional scaling (MDS) has a similar goal, but approaches the problem in a somewhat different way.

We start with observations $x_1, x_2, \dots, x_N \in \mathbb{R}^p$, and let d_{ij} be the distance between observations i and j . Often we choose Euclidean distance $d_{ij} = \|x_i - x_j\|$, but other distances may be used. Further, in some applications we may not even have available the data points x_i , but only have some *dissimilarity* measure d_{ij} (see Section 14.3.10). For example, in a wine tasting experiment, d_{ij} might be a measure of how different a subject judged wines i and j , and the subject provides such a measure for all pairs of wines i, j . MDS requires only the dissimilarities d_{ij} , in contrast to the SOM and principal curves and surfaces which need the data points x_i .

Multidimensional scaling seeks values $z_1, z_2, \dots, z_N \in \mathbb{R}^k$ to minimize the so-called *stress function*¹³

$$S_M(z_1, z_2, \dots, z_N) = \sum_{i \neq i'} (d_{ii'} - \|z_i - z_{i'}\|)^2. \quad (14.98)$$

This is known as *least squares* or *Kruskal-Shephard* scaling. The idea is to find a lower-dimensional representation of the data that preserves the pairwise distances as well as possible. Notice that the approximation is

¹²Francis Bach kindly supplied this code, and helped us set up the simulations.

¹³Some authors define stress as the square-root of S_M ; since it does not affect the optimization, we leave it squared to make comparisons with other criteria simpler.

in terms of the distances rather than squared distances (which results in slightly messier algebra). A gradient descent algorithm is used to minimize S_M .

A variation on least squares scaling is the so-called *Sammon mapping* which minimizes

$$S_{Sm}(z_1, z_2, \dots, z_N) = \sum_{i \neq i'} \frac{(d_{ii'} - \|z_i - z_{i'}\|)^2}{d_{ii'}}. \quad (14.99)$$

Here more emphasis is put on preserving smaller pairwise distances.

In *classical scaling*, we instead start with similarities $s_{ii'}$: often we use the centered inner product $s_{ii'} = \langle x_i - \bar{x}, x_{i'} - \bar{x} \rangle$. The problem then is to minimize

$$S_C(z_1, z_2, \dots, z_N) = \sum_{i, i'} (s_{ii'} - \langle z_i - \bar{z}, z_{i'} - \bar{z} \rangle)^2 \quad (14.100)$$

over $z_1, z_2, \dots, z_N \in \mathbb{R}^k$. This is attractive because there is an explicit solution in terms of eigenvectors: see Exercise 14.11. If we have distances rather than inner-products, we can convert them to centered inner-products if the distances are *Euclidean*,¹⁴ see (18.31) on page 671 in Chapter 18. If the similarities are in fact centered inner-products, classical scaling is exactly equivalent to principal components, an inherently linear dimension-reduction technique. Classical scaling is not equivalent to least squares scaling; the loss functions are different, and the mapping can be nonlinear.

Least squares and classical scaling are referred to as *metric* scaling methods, in the sense that the actual dissimilarities or similarities are approximated. *Shephard–Kruskal nonmetric scaling* effectively uses only ranks. Nonmetric scaling seeks to minimize the stress function

$$S_{NM}(z_1, z_2, \dots, z_N) = \frac{\sum_{i \neq i'} [\|z_i - z_{i'}\| - \theta(d_{ii'})]^2}{\sum_{i \neq i'} \|z_i - z_{i'}\|^2} \quad (14.101)$$

over the z_i and an arbitrary increasing function θ . With θ fixed, we minimize over z_i by gradient descent. With the z_i fixed, the method of isotonic regression is used to find the best monotonic approximation $\theta(d_{ii'})$ to $\|z_i - z_{i'}\|$. These steps are iterated until the solutions stabilize.

Like the self-organizing map and principal surfaces, multidimensional scaling represents high-dimensional data in a low-dimensional coordinate system. Principal surfaces and SOMs go a step further, and approximate the original data by a low-dimensional manifold, parametrized in the low dimensional coordinate system. In a principal surface and SOM, points

¹⁴An $N \times N$ distance matrix is Euclidean if the entries represent pairwise Euclidean distances between N points in some dimensional space.

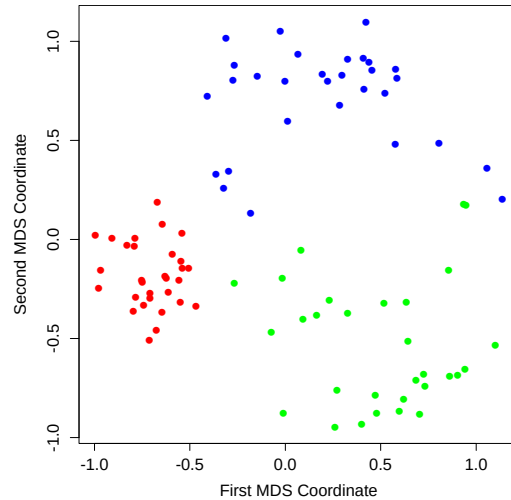


FIGURE 14.43. First two coordinates for half-sphere data, from classical multidimensional scaling.

close together in the original feature space should map close together on the manifold, but points far apart in feature space might also map close together. This is less likely in multidimensional scaling since it explicitly tries to preserve all pairwise distances.

Figure 14.43 shows the first two MDS coordinates from classical scaling for the half-sphere example. There is clear separation of the clusters, and the tighter nature of the red cluster is apparent.

14.9 Nonlinear Dimension Reduction and Local Multidimensional Scaling

Several methods have been recently proposed for nonlinear dimension reduction, similar in spirit to principal surfaces. The idea is that the data lie close to an intrinsically low-dimensional nonlinear manifold embedded in a high-dimensional space. These methods can be thought of as “flattening” the manifold, and hence reducing the data to a set of low-dimensional coordinates that represent their relative positions in the manifold. They are useful for problems where signal-to-noise ratio is very high (e.g., physical systems), and are probably not as useful for observational data with lower signal-to-noise ratios.

The basic goal is illustrated in the left panel of Figure 14.44. The data lie near a parabola with substantial curvature. Classical MDS does not pre-

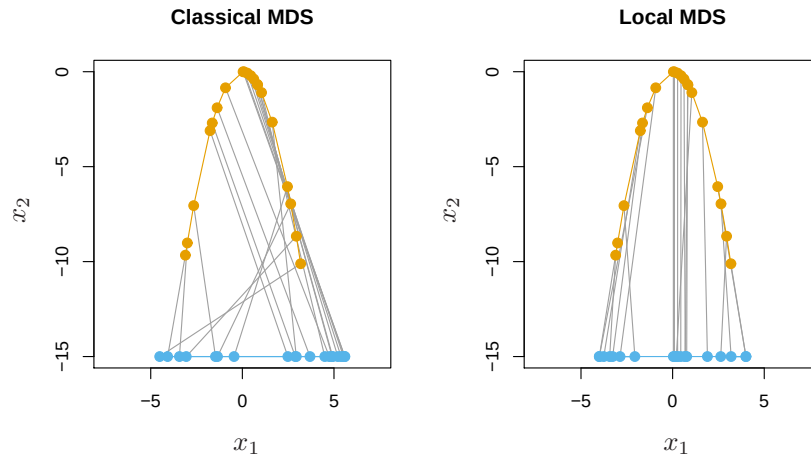


FIGURE 14.44. The orange points show data lying on a parabola, while the blue points show multidimensional scaling representations in one dimension. Classical multidimensional scaling (left panel) does not preserve the ordering of the points along the curve, because it judges points on opposite ends of the curve to be close together. In contrast, local multidimensional scaling (right panel) does a good job of preserving the ordering of the points along the curve.

serve the ordering of the points along the curve, because it judges points on opposite ends of the curve to be close together. The right panel shows the results of *local multi-dimensional scaling*, one of the three methods for non-linear multi-dimensional scaling that we discuss below. These methods use only the coordinates of the points in p dimensions, and have no other information about the manifold. Local MDS has done a good job of preserving the ordering of the points along the curve.

We now briefly describe three new approaches to nonlinear dimension reduction and manifold mapping.

Isometric feature mapping (ISOMAP) (Tenenbaum et al., 2000) constructs a graph to approximate the geodesic distance between points along the manifold. Specifically, for each data point we find its neighbors—points within some small Euclidean distance of that point. We construct a graph with an edge between any two neighboring points. The geodesic distance between any two points is then approximated by the shortest path between points on the graph. Finally, classical scaling is applied to the graph distances, to produce a low-dimensional mapping.

Local linear embedding (Roweis and Saul, 2000) takes a very different approach, trying to preserve the local affine structure of the high-dimensional data. Each data point is approximated by a linear combination of neighboring points. Then a lower dimensional representation is constructed that

best preserves these local approximations. The details are interesting, so we give them here.

1. For each data point x_i in p dimensions, we find its K -nearest neighbors $\mathcal{N}(i)$ in Euclidean distance.
2. We approximate each point by an affine mixture of the points in its neighborhood:

$$\min_{W_{ik}} \|x_i - \sum_{k \in \mathcal{N}(i)} w_{ik} x_k\|^2 \quad (14.102)$$

over weights w_{ik} satisfying $w_{ik} = 0$, $k \notin \mathcal{N}(i)$, $\sum_{k=1}^N w_{ik} = 1$. w_{ik} is the contribution of point k to the reconstruction of point i . Note that for a hope of a unique solution, we must have $K < p$.

3. Finally, we find points y_i in a space of dimension $d < p$ to minimize

$$\sum_{i=1}^N \|y_i - \sum_{k=1}^N w_{ik} y_k\|^2 \quad (14.103)$$

with w_{ik} fixed.

In step 3, we minimize

$$\text{tr}[(\mathbf{Y} - \mathbf{WY})^T(\mathbf{Y} - \mathbf{WY})] = \text{tr}[\mathbf{Y}^T(\mathbf{I} - \mathbf{W})^T(\mathbf{I} - \mathbf{W})\mathbf{Y}] \quad (14.104)$$

where $\mathbf{W}$ is $N \times N$; $\mathbf{Y}$ is $N \times d$, for some small $d < p$. The solutions $\hat{\mathbf{Y}}$ are the trailing eigenvectors of $\mathbf{M} = (\mathbf{I} - \mathbf{W})^T(\mathbf{I} - \mathbf{W})$. Since $\mathbf{1}$ is a trivial eigenvector with eigenvalue 0, we discard it and keep the next d . This has the side effect that $\mathbf{1}^T \mathbf{Y} = 0$, and hence the embedding coordinates are mean centered.

Local MDS (Chen and Buja, 2008) takes the simplest and arguably the most direct approach. We define $\mathcal{N}$ to be the symmetric set of nearby pairs of points; specifically a pair (i, i') is in $\mathcal{N}$ if point i is among the K -nearest neighbors of i' , or vice-versa. Then we construct the stress function

$$\begin{aligned} S_L(z_1, z_2, \dots, z_N) &= \sum_{(i, i') \in \mathcal{N}} (d_{ii'} - \|z_i - z_{i'}\|)^2 \\ &\quad + \sum_{(i, i') \notin \mathcal{N}} w \cdot (D - \|z_i - z_{i'}\|)^2. \end{aligned} \quad (14.105)$$

Here D is some large constant and w is a weight. The idea is that points that are not neighbors are considered to be very far apart; such pairs are given a small weight w so that they don't dominate the overall stress function. To simplify the expression, we take $w \sim 1/D$, and let $D \rightarrow \infty$. Expanding (14.105), this gives

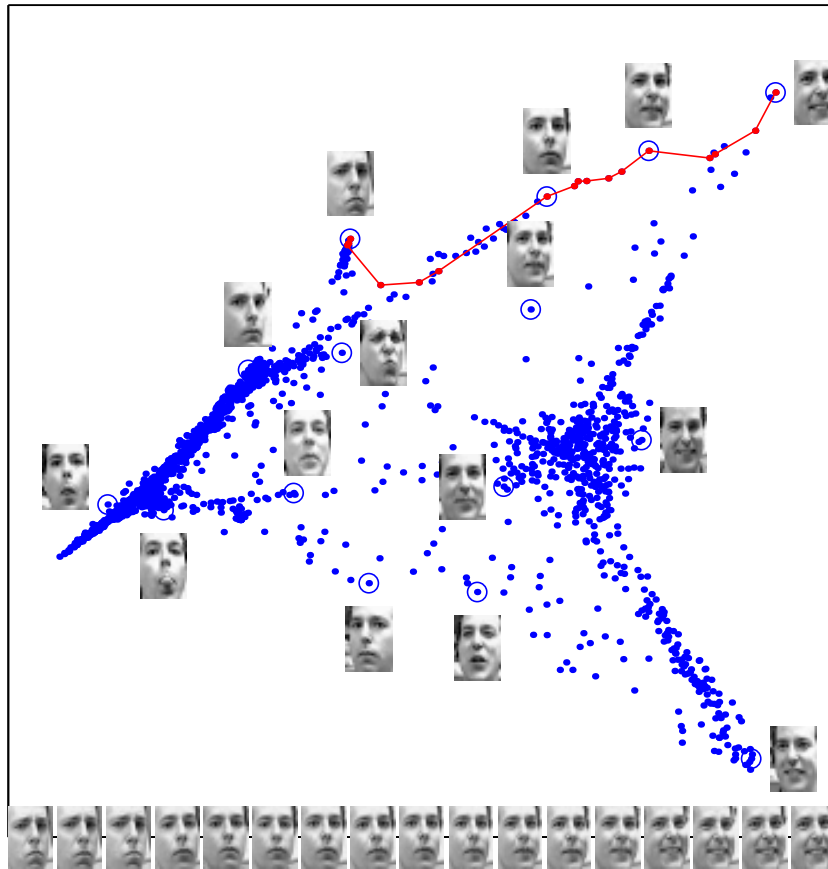


FIGURE 14.45. Images of faces mapped into the embedding space described by the first two coordinates of LLE. Next to the circled points, representative faces are shown in different parts of the space. The images at the bottom of the plot correspond to points along the top right path (linked by solid line), and illustrate one particular mode of variability in pose and expression.

$$S_L(z_1, z_2, \dots, z_N) = \sum_{(i,i') \in \mathcal{N}} (d_{ii'} - \|z_i - z_{i'}\|)^2 - \tau \sum_{(i,i') \notin \mathcal{N}} \|z_i - z_{i'}\|, \quad (14.106)$$

where $\tau = 2wD$. The first term in (14.106) tries to preserve local structure in the data, while the second term encourages the representations $z_i, z_{i'}$ for pairs (i, i') that are non-neighbors to be farther apart. Local MDS minimizes the stress function (14.106) over z_i , for fixed values of the number of neighbors K and the tuning parameter τ .

The right panel of Figure 14.44 shows the result of local MDS, using $k = 2$ neighbors and $\tau = 0.01$. We used coordinate descent with multiple starting values to find a good minimum of the (nonconvex) stress function (14.106). The ordering of the points along the curve has been largely preserved,

Figure 14.45 shows a more interesting application of one of these methods (LLE)¹⁵. The data consist of 1965 photographs, digitized as 20×28 grayscale images. The result of the first two-coordinates of LLE are shown and reveal some variability in pose and expression. Similar pictures were produced by local MDS.

In experiments reported in Chen and Buja (2008), local MDS shows superior performance, as compared to ISOMAP and LLE. They also demonstrate the usefulness of local MDS for graph layout. There are also close connections between the methods discussed here, spectral clustering (Section 14.5.3) and kernel PCA (Section 14.5.4).

14.10 The Google PageRank Algorithm

In this section we give a brief description of the original *PageRank* algorithm used by the Google search engine, an interesting recent application of unsupervised learning methods.

We suppose that we have N web pages and wish to rank them in terms of importance. For example, the N pages might all contain a string match to “statistical learning” and we might wish to rank the pages in terms of their likely relevance to a websurfer.

The *PageRank* algorithm considers a webpage to be important if many other webpages point to it. However the linking webpages that point to a given page are not treated equally: the algorithm also takes into account both the importance (*PageRank*) of the linking pages and the number of outgoing links that they have. Linking pages with higher *PageRank* are given more weight, while pages with more outgoing links are given less weight. These ideas lead to a recursive definition for *PageRank*, detailed next.

¹⁵Sam Roweis and Lawrence Saul kindly provided this figure.

Let $L_{ij} = 1$ if page j points to page i , and zero otherwise. Let $c_j = \sum_{i=1}^N L_{ij}$ equal the number of pages pointed to by page j (number of out-links). Then the Google *PageRanks* p_i are defined by the recursive relationship

$$p_i = (1 - d) + d \sum_{j=1}^N \left(\frac{L_{ij}}{c_j} \right) p_j \quad (14.107)$$

where d is a positive constant (apparently set to 0.85).

The idea is that the importance of page i is the sum of the importances of pages that point to that page. The sums are weighted by $1/c_j$, that is, each page distributes a total vote of 1 to other pages. The constant d ensures that each page gets a *PageRank* of at least $1 - d$. In matrix notation

$$\mathbf{p} = (1 - d)\mathbf{e} + d \cdot \mathbf{L}\mathbf{D}_c^{-1}\mathbf{p} \quad (14.108)$$

where $\mathbf{e}$ is a vector of N ones and $\mathbf{D}_c = \text{diag}(\mathbf{c})$ is a diagonal matrix with diagonal elements c_j . Introducing the normalization $\mathbf{e}^T \mathbf{p} = N$ (i.e., the average *PageRank* is 1), we can write (14.108) as

$$\begin{aligned} \mathbf{p} &= [(1 - d)\mathbf{e}\mathbf{e}^T/N + d\mathbf{L}\mathbf{D}_c^{-1}]\mathbf{p} \\ &= \mathbf{A}\mathbf{p} \end{aligned} \quad (14.109)$$

where the matrix $\mathbf{A}$ is the expression in square braces.

Exploiting a connection with Markov chains (see below), it can be shown that the matrix $\mathbf{A}$ has a real eigenvalue equal to one, and one is its largest eigenvalue. This means that we can find $\hat{\mathbf{p}}$ by the power method: starting with some $\mathbf{p} = \mathbf{p}_0$ we iterate

$$\mathbf{p}_k \leftarrow \mathbf{A}\mathbf{p}_{k-1}; \quad \mathbf{p}_k \leftarrow N \frac{\mathbf{p}_k}{\mathbf{e}^T \mathbf{p}_k}. \quad (14.110)$$

The fixed points $\hat{\mathbf{p}}$ are the desired *PageRanks*.

In the original paper of Page et al. (1998), the authors considered *PageRank* as a model of user behavior, where a random web surfer clicks on links at random, without regard to content. The surfer does a random walk on the web, choosing among available outgoing links at random. The factor $1 - d$ is the probability that he does not click on a link, but jumps instead to a random webpage.

Some descriptions of *PageRank* have $(1 - d)/N$ as the first term in definition (14.107), which would better coincide with the random surfer interpretation. Then the page rank solution (divided by N) is the stationary distribution of an irreducible, aperiodic Markov chain over the N webpages.

Definition (14.107) also corresponds to an irreducible, aperiodic Markov chain, with different transition probabilities than those from the $(1 - d)/N$ version. Viewing *PageRank* as a Markov chain makes clear why the matrix $\mathbf{A}$ has a maximal real eigenvalue of 1. Since $\mathbf{A}$ has positive entries with

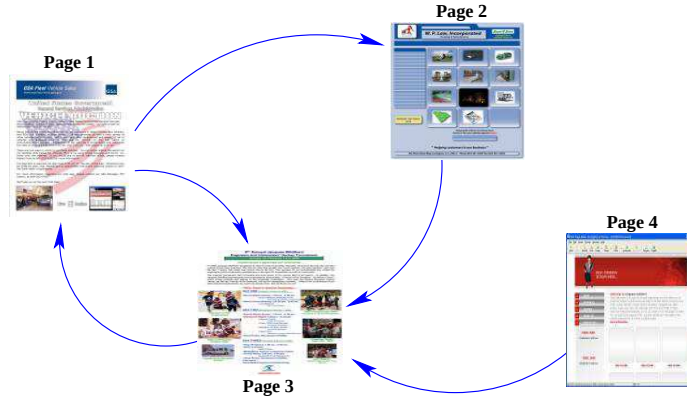


FIGURE 14.46. *PageRank algorithm: example of a small network*

each column summing to one, Markov chain theory tells us that it has a unique eigenvector with eigenvalue one, corresponding to the stationary distribution of the chain (Bremaud, 1999).

A small network is shown for illustration in Figure 14.46. The link matrix is

$$\mathbf{L} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (14.111)$$

and the number of outlinks is $\mathbf{c} = (2, 1, 1, 1)$.

The *PageRank* solution is $\hat{\mathbf{p}} = (1.49, 0.78, 1.58, 0.15)$. Notice that page 4 has no incoming links, and hence gets the minimum *PageRank* of 0.15.

Bibliographic Notes

There are many books on clustering, including Hartigan (1975), Gordon (1999) and Kaufman and Rousseeuw (1990). *K*-means clustering goes back at least to Lloyd (1957), Forgy (1965), Jancey (1966) and MacQueen (1967). Applications in engineering, especially in image compression via vector quantization, can be found in Gersho and Gray (1992). The *k*-medoid procedure is described in Kaufman and Rousseeuw (1990). Association rules are outlined in Agrawal et al. (1995). The self-organizing map was proposed by Kohonen (1989) and Kohonen (1990); Kohonen et al. (2000) give a more recent account. Principal components analysis and multidimensional scaling are described in standard books on multivariate analysis, for example, Mardia et al. (1979). Buja et al. (2008) have implemented a powerful environment called Ggvis for multidimensional scaling, and the user manual

contains a lucid overview of the subject. Figures 14.17, 14.21 (left panel) and 14.28 (left panel) were produced in Xgobi, a multidimensional data visualization package by the same authors. GGobi is a more recent implementation (Cook and Swayne, 2007). Goodall (1991) gives a technical overview of Procrustes methods in statistics, and Ramsay and Silverman (1997) discuss the shape registration problem. Principal curves and surfaces were proposed in Hastie (1984) and Hastie and Stuetzle (1989). The idea of principal points was formulated in Flury (1990), Tarpey and Flury (1996) give an exposition of the general concept of self-consistency. An excellent tutorial on spectral clustering can be found in von Luxburg (2007); this was the main source for Section 14.5.3. Luxburg credits Donath and Hoffman (1973) and Fiedler (1973) with the earliest work on the subject. A history of spectral clustering may be found in Spielman and Teng (1996). Independent component analysis was proposed by Comon (1994), with subsequent developments by Bell and Sejnowski (1995); our treatment in Section 14.7 is based on Hyvärinen and Oja (2000). Projection pursuit was proposed by Friedman and Tukey (1974), and is discussed in detail in Huber (1985). A dynamic projection pursuit algorithm is implemented in GGobi.

Exercises

Ex. 14.1 *Weights for clustering.* Show that weighted Euclidean distance

$$d_e^{(w)}(x_i, x_{i'}) = \frac{\sum_{l=1}^p w_l (x_{il} - x_{i'l})^2}{\sum_{l=1}^p w_l}$$

satisfies

$$d_e^{(w)}(x_i, x_{i'}) = d_e(z_i, z_{i'}) = \sum_{l=1}^p (z_{il} - z_{i'l})^2, \quad (14.112)$$

where

$$z_{il} = x_{il} \cdot \left(\frac{w_l}{\sum_{l=1}^p w_l} \right)^{1/2}. \quad (14.113)$$

Thus weighted Euclidean distance based on x is equivalent to unweighted Euclidean distance based on z .

Ex. 14.2 Consider a mixture model density in p -dimensional feature space,

$$g(x) = \sum_{k=1}^K \pi_k g_k(x), \quad (14.114)$$

where $g_k = N(\mu_k, \mathbf{L} \cdot \sigma^2)$ and $\pi_k \geq 0 \forall k$ with $\sum_k \pi_k = 1$. Here $\{\mu_k, \pi_k\}$, $k = 1, \dots, K$ and σ^2 are unknown parameters.

Suppose we have data $x_1, x_2, \dots, x_N \sim g(x)$ and we wish to fit the mixture model.

1. Write down the log-likelihood of the data
2. Derive an EM algorithm for computing the maximum likelihood estimates (see Section 8.1).
3. Show that if σ has a known value in the mixture model and we take $\sigma \rightarrow 0$, then in a sense this EM algorithm coincides with K -means clustering.

Ex. 14.3 In Section 14.2.6 we discuss the use of CART or PRIM for constructing generalized association rules. Show that a problem occurs with either of these methods when we generate the random data from the product-marginal distribution; i.e., by randomly permuting the values for each of the variables. Propose ways to overcome this problem.

Ex. 14.4 Cluster the demographic data of Table 14.1 using a classification tree. Specifically, generate a reference sample of the same size of the training set, by randomly permuting the values within each feature. Build a classification tree to the training sample (class 1) and the reference sample (class 0) and describe the terminal nodes having highest estimated class 1 probability. Compare the results to the PRIM results near Table 14.1 and also to the results of K -means clustering applied to the same data.

Ex. 14.5 Generate data with three features, with 30 data points in each of three classes as follows:

$$\begin{aligned}
 \theta_1 &= U(-\pi/8, \pi/8) \\
 \phi_1 &= U(0, 2\pi) \\
 x_1 &= \sin(\theta_1) \cos(\phi_1) + W_{11} \\
 y_1 &= \sin(\theta_1) \sin(\phi_1) + W_{12} \\
 z_1 &= \cos(\theta_1) + W_{13} \\
 \\
 \theta_2 &= U(\pi/2 - \pi/4, \pi/2 + \pi/4) \\
 \phi_2 &= U(-\pi/4, \pi/4) \\
 x_2 &= \sin(\theta_2) \cos(\phi_2) + W_{21} \\
 y_2 &= \sin(\theta_2) \sin(\phi_2) + W_{22} \\
 z_2 &= \cos(\theta_2) + W_{23} \\
 \\
 \theta_3 &= U(\pi/2 - \pi/4, \pi/2 + \pi/4) \\
 \phi_3 &= U(\pi/2 - \pi/4, \pi/2 + \pi/4) \\
 x_3 &= \sin(\theta_3) \cos(\phi_3) + W_{31} \\
 y_3 &= \sin(\theta_3) \sin(\phi_3) + W_{32} \\
 z_3 &= \cos(\theta_3) + W_{33}
 \end{aligned}$$

Here $U(a, b)$ indicates a uniform variate on the range $[a, b]$ and W_{jk} are independent normal variates with standard deviation 0.6. Hence the data

lie near the surface of a sphere in three clusters centered at $(1, 0, 0)$, $(0, 1, 0)$ and $(0, 0, 1)$.

Write a program to fit a SOM to these data, using the learning rates given in the text. Carry out a K -means clustering of the same data, and compare the results to those in the text.

Ex. 14.6 Write programs to implement K -means clustering and a self-organizing map (SOM), with the prototype lying on a two-dimensional grid. Apply them to the columns of the human tumor microarray data, using $K = 2, 5, 10, 20$ centroids for both. Demonstrate that as the size of the SOM neighborhood is taken to be smaller and smaller, the SOM solution becomes more similar to the K -means solution.

Ex. 14.7 Derive (14.51) and (14.52) in Section 14.5.1. Show that $\hat{\mu}$ is not unique, and characterize the family of equivalent solutions.

Ex. 14.8 Derive the solution (14.57) to the Procrustes problem (14.56). Derive also the solution to the Procrustes problem with scaling (14.58).

Ex. 14.9 Write an algorithm to solve

$$\min_{\{\beta_\ell, \mathbf{R}_\ell\}_1^L, \mathbf{M}} \sum_{\ell=1}^L \|\mathbf{X}_\ell \mathbf{R}_\ell - \mathbf{M}\|_F^2. \quad (14.115)$$

Apply it to the three S's, and compare the results to those shown in Figure 14.26.

Ex. 14.10 Derive the solution to the affine-invariant average problem (14.60). Apply it to the three S's, and compare the results to those computed in Exercise 14.9.

Ex. 14.11 *Classical multidimensional scaling.* Let $\mathbf{S}$ be the centered inner product matrix with elements $\langle x_i - \bar{x}, x_j - \bar{x} \rangle$. Let $\lambda_1 > \lambda_2 > \dots > \lambda_k$ be the k largest eigenvalues of $\mathbf{S}$, with associated eigenvectors $\mathbf{E}_k = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_k)$. Let $\mathbf{D}_k$ be a diagonal matrix with diagonal entries $\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_k}$. Show that the solutions z_i to the classical scaling problem (14.100) are the *rows* of $\mathbf{E}_k \mathbf{D}_k$.

Ex. 14.12 Consider the sparse PCA criterion (14.71).

1. Show that with Θ fixed, solving for $\mathbf{V}$ amounts to K separate elastic-net regression problems, with responses the K elements of $\Theta^T x_i$.
2. Show that with $\mathbf{V}$ fixed, solving for Θ amounts to a reduced-rank version of the Procrustes problem, which reduces to

$$\max_{\Theta} \text{trace}(\Theta^T \mathbf{M}) \text{ subject to } \Theta^T \Theta = \mathbf{I}_K, \quad (14.116)$$

where $\mathbf{M}$ and Θ are both $p \times K$ with $K \leq p$. If $\mathbf{M} = \mathbf{U} \mathbf{D} \mathbf{Q}^T$ is the SVD of $\mathbf{M}$, show that the optimal $\Theta = \mathbf{U} \mathbf{Q}^T$.

Ex. 14.13 Generate 200 data points with three features, lying close to a *helix*. In detail, define $X_1 = \cos(s) + 0.1 \cdot Z_1$, $X_2 = \sin(s) + 0.1 \cdot Z_2$, $X_3 = s + 0.1 \cdot Z_3$ where s takes on 200 equally spaced values between 0 and 2π , and Z_1, Z_2, Z_3 are independent and have standard Gaussian distributions.

- (a) Fit a principal curve to the data and plot the estimated coordinate functions. Compare them to the underlying functions $\cos(s), \sin(s)$ and s .
- (b) Fit a self-organizing map to the same data, and see if you can discover the helical shape of the original point cloud.

Ex. 14.14 Pre- and post-multiply equation (14.81) by a diagonal matrix containing the inverse variances of the X_j . Hence obtain an equivalent decomposition for the correlation matrix, in the sense that a simple scaling is applied to the matrix $\mathbf{A}$.

Ex. 14.15 Generate 200 observations of three variates X_1, X_2, X_3 according to

$$\begin{aligned} X_1 &\sim Z_1 \\ X_2 &= X_1 + 0.001 \cdot Z_2 \\ X_3 &= 10 \cdot Z_3 \end{aligned} \tag{14.117}$$

where Z_1, Z_2, Z_3 are independent standard normal variates. Compute the leading principal component and factor analysis directions. Hence show that the leading principal component aligns itself in the maximal variance direction X_3 , while the leading factor essentially ignores the uncorrelated component X_3 , and picks up the correlated component $X_2 + X_1$ (Geoffrey Hinton, personal communication).

Ex. 14.16 Consider the kernel principal component procedure outlined in Section 14.5.4. Argue that the number M of principal components is equal to the rank of $\mathbf{K}$, which is the number of non-zero elements in $\mathbf{D}$. Show that the m th component $\mathbf{z}_m$ (m th column of $\mathbf{Z}$) can be written (up to centering) as $z_{im} = \sum_{j=1}^N \alpha_{jm} K(x_i, x_j)$, where $\alpha_{jm} = u_{jm}/d_m$. Show that the mapping of a new observation x_0 to the m th component is given by $z_{0m} = \sum_{j=1}^N \alpha_{jm} K(x_0, x_j)$.

Ex. 14.17 Show that with $g_1(x) = \sum_{j=1}^N c_j K(x, x_j)$, the solution to (14.66) is given by $\hat{c}_j = u_{j1}/d_1$, where $\mathbf{u}_1$ is the first column of $\mathbf{U}$ in (14.65), and d_1 the first diagonal element of $\mathbf{D}$. Show that the second and subsequent principal component functions are defined in a similar manner (*hint*: see Section 5.8.1.)

Ex. 14.18 Consider the regularized log-likelihood for the density estimation problem arising in ICA,

$$\frac{1}{N} \sum_{i=1}^N [\log \phi(s_i) + g(s_i)] - \int \phi(t) e^{g(t)} dt - \lambda \int \{g'''(t)\}^2(t) dt. \quad (14.118)$$

The solution $\hat{g}$ is a quartic smoothing spline, and can be written as $\hat{g}(s) = \hat{q}(s) + \hat{q}_\perp(s)$, where q is a quadratic function (in the null space of the penalty). Let $q(s) = \theta_0 + \theta_1 s + \theta_2 s^2$. By examining the stationarity conditions for $\hat{\theta}_k$, $k = 1, 2, 3$, show that the solution $\hat{f} = \phi e^{\hat{g}}$ is a density, and has mean zero and variance one. If we used a second-derivative penalty $\int \{g''(t)\}^2(t) dt$ instead, what simple modification could we make to the problem to maintain the three moment conditions?

Ex. 14.19 If $\mathbf{A}$ is $p \times p$ orthogonal, show that the first term in (14.91) on page 567

$$\sum_{j=1}^p \sum_{i=1}^N \log \phi(a_j^T x_i),$$

with a_j the j th column of $\mathbf{A}$, does not depend on $\mathbf{A}$.

Ex. 14.20 *Fixed point algorithm for ICA* (Hyvärinen et al., 2001). Consider maximizing $C(a) = E\{g(a^T X)\}$ with respect to a , with $\|a\| = 1$ and $\text{Cov}(X) = I$. Use a Lagrange multiplier to enforce the norm constraint, and write down the first two derivatives of the modified criterion. Use the approximation

$$E\{X X^T g''(a^T X)\} \approx E\{X X^T\} E\{g''(a^T X)\}$$

to show that the Newton update can be written as the fixed-point update (14.96).

Ex. 14.21 Consider an undirected graph with non-negative edge weights $w_{ii'}$ and graph Laplacian $\mathbf{L}$. Suppose there are m connected components $A_1, A_2, \dots, A_m$ in the graph. Show that there are m eigenvectors of $\mathbf{L}$ corresponding to eigenvalue zero, and the indicator vectors of these components $I_{A_1}, I_{A_2}, \dots, I_{A_m}$ span the zero eigenspace.

Ex. 14.22

- Show that definition (14.108) implies that the sum of the *PageRanks* p_i is N , the number of web pages.
- Write a program to compute the *PageRank* solutions by the power method using formulation (14.107). Apply it to the network of Figure 14.47.

Ex. 14.23 *Algorithm for non-negative matrix factorization* (Wu and Lange, 2007). A function $g(x, y)$ is said to *minorize* a function $f(x)$ if

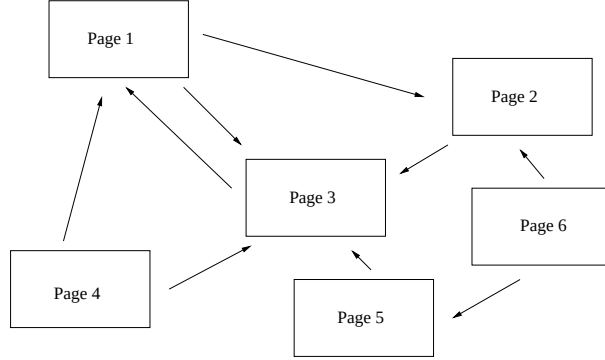


FIGURE 14.47. Example of a small network.

$$g(x, y) \leq f(x), \quad g(x, x) = f(x) \quad (14.119)$$

for all x, y in the domain. This is useful for maximizing $f(x)$ since it is easy to show that $f(x)$ is nondecreasing under the update

$$x^{s+1} = \operatorname{argmax}_x g(x, x^s) \quad (14.120)$$

There are analogous definitions for *majorization*, for minimizing a function $f(x)$. The resulting algorithms are known as *MM* algorithms, for “minorize-maximize” or “majorize-minimize” (Lange, 2004). It also can be shown that the EM algorithm (8.5) is an example of an MM algorithm: see Section 8.5.3 and Exercise 8.2 for details.

- (a) Consider maximization of the function $L(\mathbf{W}, \mathbf{H})$ in (14.73), written here without the matrix notation

$$L(\mathbf{W}, \mathbf{H}) = \sum_{i=1}^N \sum_{j=1}^p \left[x_{ij} \log \left(\sum_{k=1}^r w_{ik} h_{kj} \right) - \sum_{k=1}^r w_{ik} h_{kj} \right].$$

Using the concavity of $\log(x)$, show that for any set of r values $y_k \geq 0$ and $0 \leq c_k \leq 1$ with $\sum_{k=1}^r c_k = 1$,

$$\log \left(\sum_{k=1}^r y_k \right) \geq \sum_{k=1}^r c_k \log(y_k / c_k)$$

Hence

$$\log \left(\sum_{k=1}^r w_{ik} h_{kj} \right) \geq \sum_{k=1}^r \frac{a_{ikj}^s}{b_{ij}^s} \log \left(\frac{b_{ij}^s}{a_{ikj}^s} w_{ik} h_{kj} \right),$$

where

$$a_{ikj}^s = w_{ik}^s h_{kj}^s \text{ and } b_{ij}^s = \sum_{k=1}^r w_{ik}^s h_{kj}^s,$$

and s indicates the current iteration.

(b) Hence show that, ignoring constants, the function

$$g(\mathbf{W}, \mathbf{H} \mid \mathbf{W}^s, \mathbf{H}^s) = \sum_{i=1}^N \sum_{j=1}^p \sum_{k=1}^r x_{ij} \frac{a_{ikj}^s}{b_{ij}^s} (\log w_{ik} + \log h_{kj}) - \sum_{i=1}^N \sum_{j=1}^p \sum_{k=1}^r w_{ik} h_{kj}$$

minorizes $L(\mathbf{W}, \mathbf{H})$.

(c) Set the partial derivatives of $g(\mathbf{W}, \mathbf{H} \mid \mathbf{W}^s, \mathbf{H}^s)$ to zero and hence derive the updating steps (14.74).

Ex. 14.24 Consider the non-negative matrix factorization (14.72) in the rank one case ($r = 1$).

(a) Show that the updates (14.74) reduce to

$$\begin{aligned} w_i &\leftarrow w_i \frac{\sum_{j=1}^p x_{ij}}{\sum_{j=1}^p w_i h_j} \\ h_j &\leftarrow h_j \frac{\sum_{i=1}^N x_{ij}}{\sum_{i=1}^N w_i h_j} \end{aligned} \quad (14.121)$$

where $w_i = w_{i1}$, $h_j = h_{1j}$. This is an example of the *iterative proportional scaling* procedure, applied to the independence model for a two-way contingency table (Fienberg, 1977, for example).

(b) Show that the final iterates have the explicit form

$$w_i = c \cdot \frac{\sum_{j=1}^p x_{ij}}{\sum_{i=1}^N \sum_{j=1}^p x_{ij}}, \quad h_k = \frac{1}{c} \cdot \frac{\sum_{i=1}^N x_{ik}}{\sum_{i=1}^N \sum_{j=1}^p x_{ij}} \quad (14.122)$$

for any constant $c > 0$. These are equivalent to the usual row and column estimates for a two-way independence model.

Ex. 14.25 Fit a non-negative matrix factorization model to the collection of two's in the digits database. Use 25 basis elements, and compare with a 24- component (plus mean) PCA model. In both cases display the $\mathbf{W}$ and $\mathbf{H}$ matrices as in Figure 14.33.

15

Random Forests

15.1 Introduction

Bagging or *bootstrap aggregation* (section 8.7) is a technique for reducing the variance of an estimated prediction function. Bagging seems to work especially well for high-variance, low-bias procedures, such as trees. For regression, we simply fit the same regression tree many times to bootstrap-sampled versions of the training data, and average the result. For classification, a *committee* of trees each cast a vote for the predicted class.

Boosting in Chapter 10 was initially proposed as a committee method as well, although unlike bagging, the committee of *weak learners* evolves over time, and the members cast a weighted vote. Boosting appears to dominate bagging on most problems, and became the preferred choice.

Random forests (Breiman, 2001) is a substantial modification of bagging that builds a large collection of *de-correlated* trees, and then averages them. On many problems the performance of random forests is very similar to boosting, and they are simpler to train and tune. As a consequence, random forests are popular, and are implemented in a variety of packages.

15.2 Definition of Random Forests

The essential idea in bagging (Section 8.7) is to average many noisy but approximately unbiased models, and hence reduce the variance. Trees are ideal candidates for bagging, since they can capture complex interaction

Algorithm 15.1 *Random Forest for Regression or Classification.*

1. For $b = 1$ to B :
 - (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
 - (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
 - i. Select m variables at random from the p variables.
 - ii. Pick the best variable/split-point among the m .
 - iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x :

Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$.

Classification: Let $\hat{C}_b(x)$ be the class prediction of the b th random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \text{majority vote } \{\hat{C}_b(x)\}_1^B$.

structures in the data, and if grown sufficiently deep, have relatively low bias. Since trees are notoriously noisy, they benefit greatly from the averaging. Moreover, since each tree generated in bagging is identically distributed (i.d.), the expectation of an average of B such trees is the same as the expectation of any one of them. This means the bias of bagged trees is the same as that of the individual (bootstrap) trees, and the only hope of improvement is through variance reduction. This is in contrast to boosting, where the trees are grown in an adaptive way to remove bias, and hence are not i.d.

An average of B i.i.d. random variables, each with variance σ^2 , has variance $\frac{1}{B}\sigma^2$. If the variables are simply i.d. (identically distributed, but not necessarily independent) with positive pairwise correlation ρ , the variance of the average is (Exercise 15.1)

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \quad (15.1)$$

As B increases, the second term disappears, but the first remains, and hence the size of the correlation of pairs of bagged trees limits the benefits of averaging. The idea in random forests (Algorithm 15.1) is to improve the variance reduction of bagging by reducing the correlation between the trees, without increasing the variance too much. This is achieved in the tree-growing process through random selection of the input variables.

Specifically, when growing a tree on a bootstrapped dataset:

Before each split, select $m \leq p$ of the input variables at random as candidates for splitting.

Typically values for m are $\sqrt{p}$ or even as low as 1.

After B such trees $\{T(x; \Theta_b)\}_1^B$ are grown, the random forest (regression) predictor is

$$\hat{f}_{\text{rf}}^B(x) = \frac{1}{B} \sum_{b=1}^B T(x; \Theta_b). \quad (15.2)$$

As in Section 10.9 (page 356), Θ_b characterizes the b th random forest tree in terms of split variables, cutpoints at each node, and terminal-node values. Intuitively, reducing m will reduce the correlation between any pair of trees in the ensemble, and hence by (15.1) reduce the variance of the average.

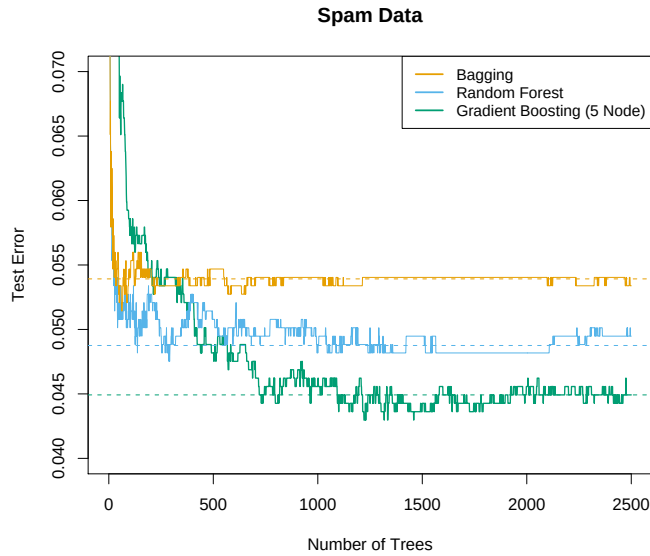


FIGURE 15.1. Bagging, random forest, and gradient boosting, applied to the spam data. For boosting, 5-node trees were used, and the number of trees were chosen by 10-fold cross-validation (2500 trees). Each “step” in the figure corresponds to a change in a single misclassification (in a test set of 1536).

Not all estimators can be improved by shaking up the data like this. It seems that highly nonlinear estimators, such as trees, benefit the most. For bootstrapped trees, ρ is typically small (0.05 or lower is typical; see Figure 15.9), while σ^2 is not much larger than the variance for the original tree. On the other hand, bagging does not change *linear* estimates, such as the sample mean (hence its variance either); the pairwise correlation between bootstrapped means is about 50% (Exercise 15.4).

Random forests are popular. Leo Breiman's¹ collaborator Adele Cutler maintains a random forest website² where the software is freely available, with more than 3000 downloads reported by 2002. There is a `randomForest` package in R, maintained by Andy Liaw, available from the CRAN website.

The authors make grand claims about the success of random forests: “most accurate,” “most interpretable,” and the like. In our experience random forests do remarkably well, with very little tuning required. A random forest classifier achieves 4.88% misclassification error on the `spam` test data, which compares well with all other methods, and is not significantly worse than gradient boosting at 4.5%. Bagging achieves 5.4% which is significantly worse than either (using the McNemar test outlined in Exercise 10.6), so it appears on this example the additional randomization helps.

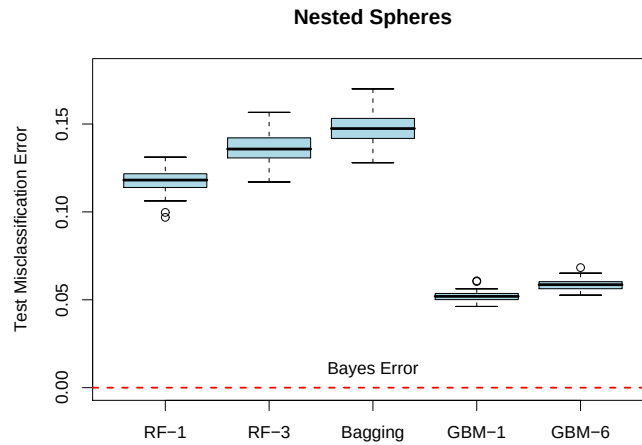


FIGURE 15.2. The results of 50 simulations from the “nested spheres” model in $\mathbb{R}^{10}$. The Bayes decision boundary is the surface of a sphere (additive). “RF-3” refers to a random forest with $m = 3$, and “GBM-6” a gradient boosted model with interaction order six; similarly for “RF-1” and “GBM-1.” The training sets were of size 2000, and the test sets 10,000.

Figure 15.1 shows the test-error progression on 2500 trees for the three methods. In this case there is some evidence that gradient boosting has started to overfit, although 10-fold cross-validation chose all 2500 trees.

¹Sadly, Leo Breiman died in July, 2005.

²<http://www.math.usu.edu/~adele/forests/>

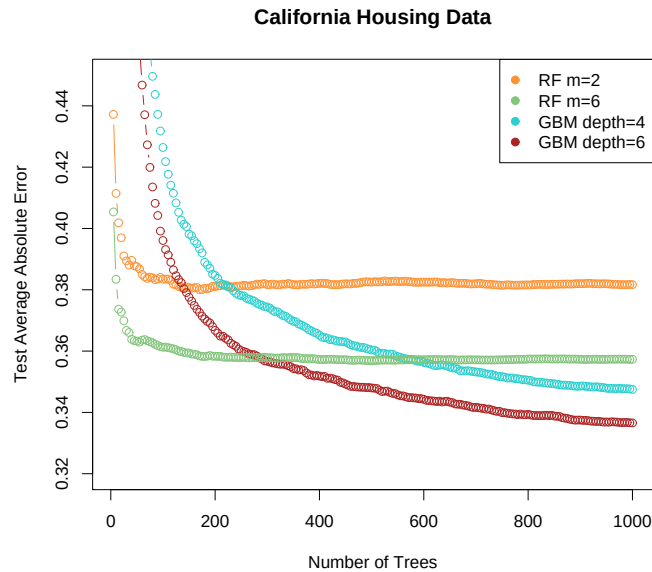


FIGURE 15.3. Random forests compared to gradient boosting on the California housing data. The curves represent mean absolute error on the test data as a function of the number of trees in the models. Two random forests are shown, with $m = 2$ and $m = 6$. The two gradient boosted models use a shrinkage parameter $\nu = 0.05$ in (10.41), and have interaction depths of 4 and 6. The boosted models outperform random forests.

Figure 15.2 shows the results of a simulation³ comparing random forests to gradient boosting on the *nested spheres* problem [Equation (10.2) in Chapter 10]. Boosting easily outperforms random forests here. Notice that smaller m is better here, although part of the reason could be that the true decision boundary is additive.

Figure 15.3 compares random forests to boosting (with shrinkage) in a regression problem, using the California housing data (Section 10.14.1). Two strong features that emerge are

- Random forests stabilize at about 200 trees, while at 1000 trees boosting continues to improve. Boosting is slowed down by the shrinkage, as well as the fact that the trees are much smaller.
- Boosting outperforms random forests here. At 1000 terms, the weaker boosting model (GBM depth 4) has a smaller error than the stronger

³Details: The random forests were fit using the R package `randomForest` 4.5-11, with 500 trees. The gradient boosting models were fit using R package `gbm` 1.5, with shrinkage parameter set to 0.05, and 2000 trees.

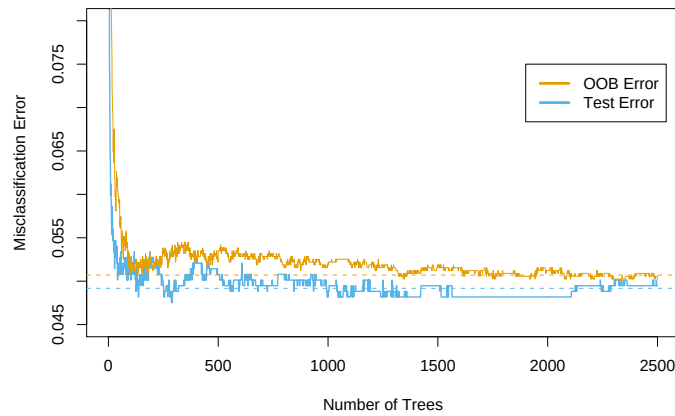


FIGURE 15.4. OOB error computed on the `spam` training data, compared to the test error computed on the test set.

random forest (RF $m = 6$); a Wilcoxon test on the mean differences in absolute errors has a p-value of 0.007. For larger m the random forests performed no better.

15.3 Details of Random Forests

We have glossed over the distinction between random forests for classification versus regression. When used for classification, a random forest obtains a class vote from each tree, and then classifies using majority vote (see Section 8.7 on bagging for a similar discussion). When used for regression, the predictions from each tree at a target point x are simply averaged, as in (15.2). In addition, the inventors make the following recommendations:

- For classification, the default value for m is $\lfloor \sqrt{p} \rfloor$ and the minimum node size is one.
- For regression, the default value for m is $\lfloor p/3 \rfloor$ and the minimum node size is five.

In practice the best values for these parameters will depend on the problem, and they should be treated as tuning parameters. In Figure 15.3 $m = 6$ performs much better than the default value $\lfloor 8/3 \rfloor = 2$.

15.3.1 Out of Bag Samples

An important feature of random forests is its use of *out-of-bag* (OOB) samples:

For each observation $z_i = (x_i, y_i)$, construct its random forest predictor by averaging only those trees corresponding to bootstrap samples in which z_i did not appear.

An OOB error estimate is almost identical to that obtained by N -fold cross-validation; see Exercise 15.2. Hence unlike many other nonlinear estimators, random forests can be fit in one sequence, with cross-validation being performed along the way. Once the OOB error stabilizes, the training can be terminated.

Figure 15.4 shows the OOB misclassification error for the `spam` data, compared to the test error. Although 2500 trees are averaged here, it appears from the plot that about 200 would be sufficient.

15.3.2 Variable Importance

Variable importance plots can be constructed for random forests in exactly the same way as they were for gradient-boosted models (Section 10.13). At each split in each tree, the improvement in the split-criterion is the importance measure attributed to the splitting variable, and is accumulated over all the trees in the forest separately for each variable. The left plot of Figure 15.5 shows the variable importances computed in this way for the `spam` data; compare with the corresponding Figure 10.6 on page 354 for gradient boosting. Boosting ignores some variables completely, while the random forest does not. The candidate split-variable selection increases the chance that any single variable gets included in a random forest, while no such selection occurs with boosting.

Random forests also use the OOB samples to construct a different *variable-importance* measure, apparently to measure the prediction strength of each variable. When the b th tree is grown, the OOB samples are passed down the tree, and the prediction accuracy is recorded. Then the values for the j th variable are randomly permuted in the OOB samples, and the accuracy is again computed. The decrease in accuracy as a result of this permuting is averaged over all trees, and is used as a measure of the importance of variable j in the random forest. These are expressed as a percent of the maximum in the right plot in Figure 15.5. Although the rankings of the two methods are similar, the importances in the right plot are more uniform over the variables. The randomization effectively voids the effect of a variable, much like setting a coefficient to zero in a linear model (Exercise 15.7). This does not measure the effect on prediction were this variable not available, because if the model was refitted without the variable, other variables could be used as surrogates.

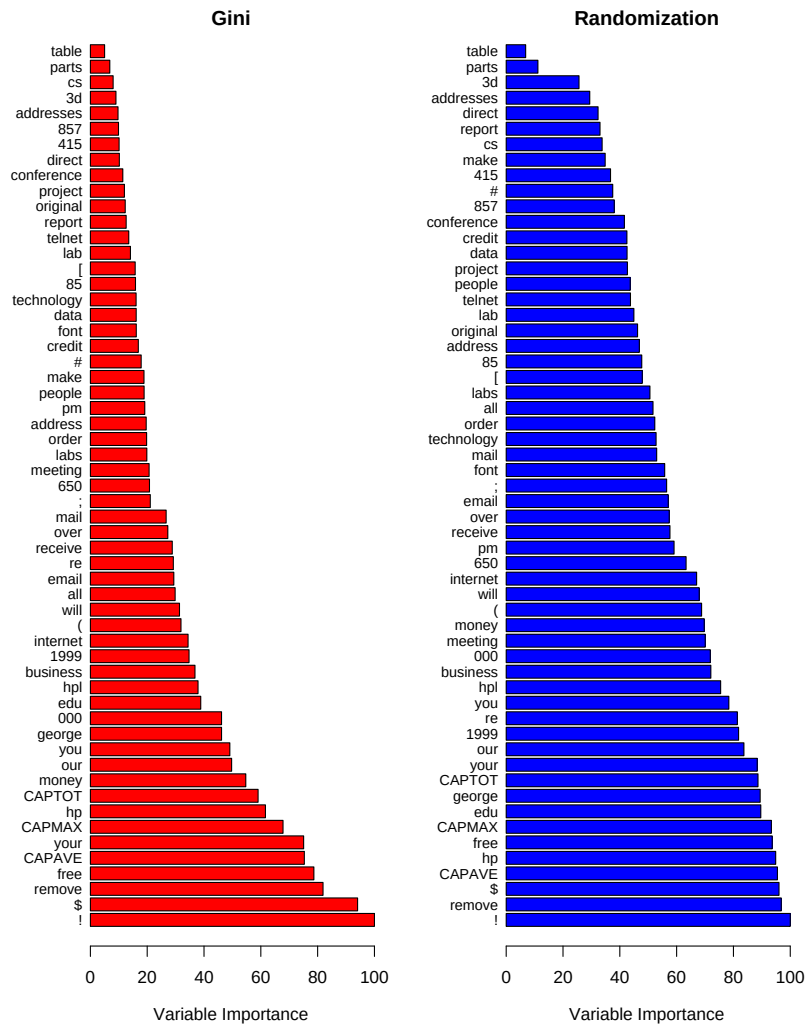


FIGURE 15.5. Variable importance plots for a classification random forest grown on the `spam` data. The left plot bases the importance on the Gini splitting index, as in gradient boosting. The rankings compare well with the rankings produced by gradient boosting (Figure 10.6 on page 354). The right plot uses OOB randomization to compute variable importances, and tends to spread the importances more uniformly.

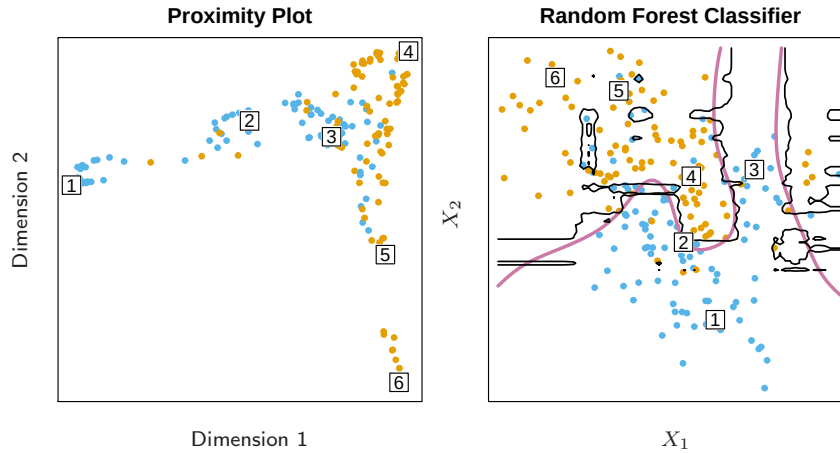


FIGURE 15.6. (Left): Proximity plot for a random forest classifier grown to the mixture data. (Right): Decision boundary and training data for random forest on mixture data. Six points have been identified in each plot.

15.3.3 Proximity Plots

One of the advertised outputs of a random forest is a *proximity plot*. Figure 15.6 shows a proximity plot for the mixture data defined in Section 2.3.3 in Chapter 2. In growing a random forest, an $N \times N$ proximity matrix is accumulated for the training data. For every tree, any pair of OOB observations sharing a terminal node has their proximity increased by one. This proximity matrix is then represented in two dimensions using multidimensional scaling (Section 14.8). The idea is that even though the data may be high-dimensional, involving mixed variables, etc., the proximity plot gives an indication of which observations are effectively close together in the eyes of the random forest classifier.

Proximity plots for random forests often look very similar, irrespective of the data, which casts doubt on their utility. They tend to have a star shape, one arm per class, which is more pronounced the better the classification performance.

Since the mixture data are two-dimensional, we can map points from the proximity plot to the original coordinates, and get a better understanding of what they represent. It seems that points in pure regions class-wise map to the extremities of the star, while points nearer the decision boundaries map nearer the center. This is not surprising when we consider the construction of the proximity matrices. Neighboring points in pure regions will often end up sharing a bucket, since when a terminal node is pure, it is no longer

split by a random forest tree-growing algorithm. On the other hand, pairs of points that are close but belong to different classes will sometimes share a terminal node, but not always.

15.3.4 Random Forests and Overfitting

When the number of variables is large, but the fraction of relevant variables small, random forests are likely to perform poorly with small m . At each split the chance can be small that the relevant variables will be selected. Figure 15.7 shows the results of a simulation that supports this claim. Details are given in the figure caption and Exercise 15.3. At the top of each pair we see the hyper-geometric probability that a relevant variable will be selected at any split by a random forest tree (in this simulation, the relevant variables are all equal in stature). As this probability gets small, the gap between boosting and random forests increases. When the number of relevant variables increases, the performance of random forests is surprisingly robust to an increase in the number of noise variables. For example, with 6 relevant and 100 noise variables, the probability of a relevant variable being selected at any split is 0.46, assuming $m = \sqrt{6 + 100} \approx 10$. According to Figure 15.7, this does not hurt the performance of random forests compared with boosting. This robustness is largely due to the relative insensitivity of misclassification cost to the bias and variance of the probability estimates in each tree. We consider random forests for regression in the next section.

Another claim is that random forests “cannot overfit” the data. It is certainly true that increasing B does not cause the random forest sequence to overfit; like bagging, the random forest estimate (15.2) approximates the expectation

$$\hat{f}_{\text{rf}}(x) = \mathbb{E}_{\Theta} T(x; \Theta) = \lim_{B \rightarrow \infty} \hat{f}(x)_{\text{rf}}^B \quad (15.3)$$

with an average over B realizations of Θ . The distribution of Θ here is conditional on the training data. However, *this limit can overfit the data*; the average of fully grown trees can result in too rich a model, and incur unnecessary variance. Segal (2004) demonstrates small gains in performance by controlling the depths of the individual trees grown in random forests. Our experience is that using full-grown trees seldom costs much, and results in one less tuning parameter.

Figure 15.8 shows the modest effect of depth control in a simple regression example. Classifiers are less sensitive to variance, and this effect of overfitting is seldom seen with random-forest classification.

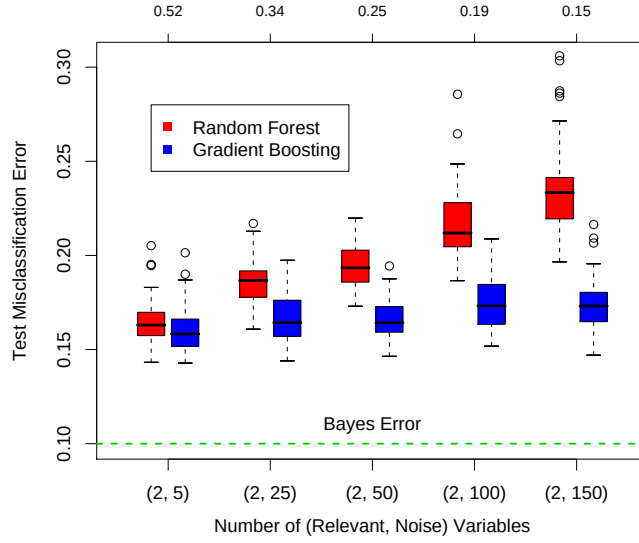


FIGURE 15.7. A comparison of random forests and gradient boosting on problems with increasing numbers of noise variables. In each case the true decision boundary depends on two variables, and an increasing number of noise variables are included. Random forests uses its default value $m = \sqrt{p}$. At the top of each pair is the probability that one of the relevant variables is chosen at any split. The results are based on 50 simulations for each pair, with a training sample of 300, and a test sample of 500. See Exercise 15.3.

15.4 Analysis of Random Forests



In this section we analyze the mechanisms at play with the additional randomization employed by random forests. For this discussion we focus on regression and squared error loss, since this gets at the main points, and bias and variance are more complex with 0–1 loss (see Section 7.3.1). Furthermore, even in the case of a classification problem, we can consider the random-forest average as an estimate of the class posterior probabilities, for which bias and variance are appropriate descriptors.

15.4.1 Variance and the De-Correlation Effect

The limiting form ($B \rightarrow \infty$) of the random forest regression estimator is

$$\hat{f}_{\text{rf}}(x) = E_{\Theta|\mathbf{Z}} T(x; \Theta(\mathbf{Z})), \quad (15.4)$$

where we have made explicit the dependence on the training data $\mathbf{Z}$. Here we consider estimation at a single target point x . From (15.1) we see that

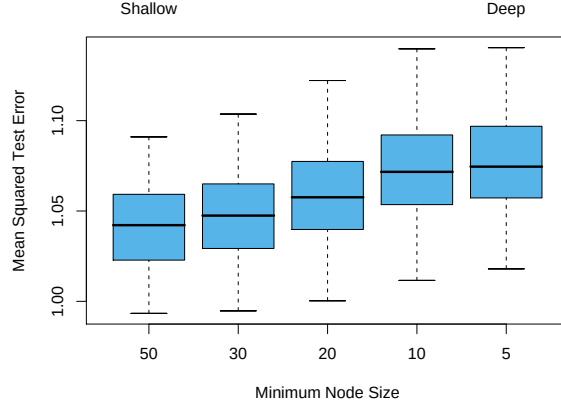


FIGURE 15.8. The effect of tree size on the error in random forest regression. In this example, the true surface was additive in two of the 12 variables, plus additive unit-variance Gaussian noise. Tree depth is controlled here by the minimum node size; the smaller the minimum node size, the deeper the trees.

$$\text{Var} \hat{f}_{\text{rf}}(x) = \rho(x) \sigma^2(x). \quad (15.5)$$

Here

- $\rho(x)$ is the *sampling* correlation between any pair of trees used in the averaging:

$$\rho(x) = \text{corr}[T(x; \Theta_1(\mathbf{Z})), T(x; \Theta_2(\mathbf{Z}))], \quad (15.6)$$

where $\Theta_1(\mathbf{Z})$ and $\Theta_2(\mathbf{Z})$ are a randomly drawn pair of random forest trees grown to the randomly sampled $\mathbf{Z}$;

- $\sigma^2(x)$ is the sampling variance of any single randomly drawn tree,

$$\sigma^2(x) = \text{Var} T(x; \Theta(\mathbf{Z})). \quad (15.7)$$

It is easy to confuse $\rho(x)$ with the average correlation between fitted trees in a *given* random-forest ensemble; that is, think of the fitted trees as N -vectors, and compute the average pairwise correlation between these vectors, conditioned on the data. This is *not* the case; this conditional correlation is not directly relevant in the averaging process, and the dependence on x in $\rho(x)$ warns us of the distinction. Rather, $\rho(x)$ is the theoretical correlation between a pair of random-forest trees evaluated at x , induced by repeatedly making training sample draws $\mathbf{Z}$ from the population, and then drawing a pair of random forest trees. In statistical jargon, this is the correlation induced by the *sampling distribution* of $\mathbf{Z}$ and Θ .

More precisely, the variability averaged over in the calculations in (15.6) and (15.7) is both

- conditional on $\mathbf{Z}$: due to the bootstrap sampling and feature sampling at each split, and
- a result of the sampling variability of $\mathbf{Z}$ itself.

In fact, the conditional covariance of a pair of tree fits at x is zero, because the bootstrap and feature sampling is i.i.d; see Exercise 15.5.

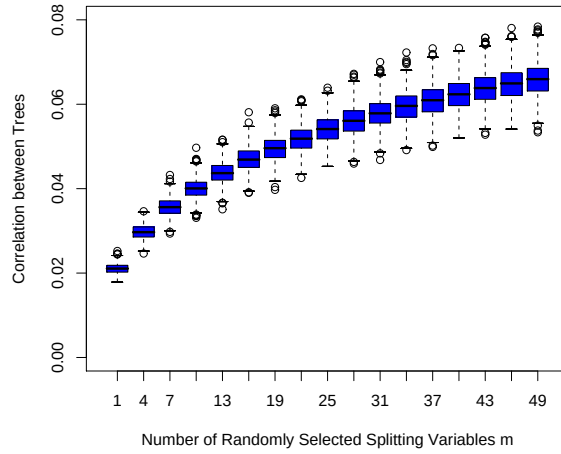


FIGURE 15.9. Correlations between pairs of trees drawn by a random-forest regression algorithm, as a function of m . The boxplots represent the correlations at 600 randomly chosen prediction points x .

The following demonstrations are based on a simulation model

$$Y = \frac{1}{\sqrt{50}} \sum_{j=1}^{50} X_j + \varepsilon, \quad (15.8)$$

with all the X_j and ε iid Gaussian. We use 500 training sets of size 100, and a single set of test locations of size 600. Since regression trees are nonlinear in $\mathbf{Z}$, the patterns we see below will differ somewhat depending on the structure of the model.

Figure 15.9 shows how the correlation (15.6) between pairs of trees decreases as m decreases: pairs of tree predictions at x for different training sets $\mathbf{Z}$ are likely to be less similar if they do not use the same splitting variables.

In the left panel of Figure 15.10 we consider the variances of single tree predictors, $\text{Var}T(x; \Theta(\mathbf{Z}))$ (averaged over 600 prediction points x drawn randomly from our simulation model). This is the total variance, and can be

decomposed into two parts using standard conditional variance arguments (see Exercise 15.5):

$$\begin{aligned} \text{Var}_{\Theta, \mathbf{Z}} T(x; \Theta(\mathbf{Z})) &= \text{Var}_{\mathbf{Z}} \mathbb{E}_{\Theta|\mathbf{Z}} T(x; \Theta(\mathbf{Z})) + \mathbb{E}_{\mathbf{Z}} \text{Var}_{\Theta|\mathbf{Z}} T(x; \Theta(\mathbf{Z})) \\ \text{Total Variance} &= \text{Var}_{\mathbf{Z}} \hat{f}_{\text{rf}}(x) + \text{within-}\mathbf{Z} \text{ Variance} \end{aligned} \quad (15.9)$$

The second term is the within- $\mathbf{Z}$ variance—a result of the randomization, which increases as m decreases. The first term is in fact the sampling variance of the random forest ensemble (shown in the right panel), which decreases as m decreases. The variance of the individual trees does not change appreciably over much of the range of m , hence in light of (15.5), the variance of the ensemble is dramatically lower than this tree variance.

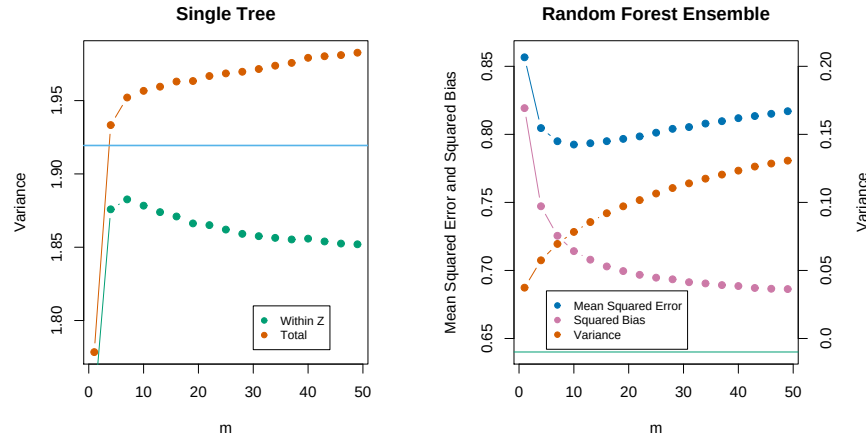


FIGURE 15.10. Simulation results. The left panel shows the average variance of a single random forest tree, as a function of m . “Within $\mathbf{Z}$ ” refers to the average within-sample contribution to the variance, resulting from the bootstrap sampling and split-variable sampling (15.9). “Total” includes the sampling variability of $\mathbf{Z}$. The horizontal line is the average variance of a single fully grown tree (without bootstrap sampling). The right panel shows the average mean-squared error, squared bias and variance of the ensemble, as a function of m . Note that the variance axis is on the right (same scale, different level). The horizontal line is the average squared-bias of a fully grown tree.

15.4.2 Bias

As in bagging, the bias of a random forest is the same as the bias of any of the individual sampled trees $T(x; \Theta(\mathbf{Z}))$:

$$\begin{aligned}
\text{Bias}(x) &= \mu(x) - \mathbb{E}_{\mathbf{Z}} \hat{f}_{\text{rf}}(x) \\
&= \mu(x) - \mathbb{E}_{\mathbf{Z}} \mathbb{E}_{\Theta|\mathbf{Z}} T(x; \Theta(\mathbf{Z})).
\end{aligned} \tag{15.10}$$

This is also typically greater (in absolute terms) than the bias of an unpruned tree grown to $\mathbf{Z}$, since the randomization and reduced sample space impose restrictions. Hence the improvements in prediction obtained by bagging or random forests are *solely a result of variance reduction*.

Any discussion of bias depends on the unknown true function. Figure 15.10 (right panel) shows the squared bias for our additive model simulation (estimated from the 500 realizations). Although for different models the shape and rate of the bias curves may differ, the general trend is that as m decreases, the bias increases. Shown in the figure is the mean-squared error, and we see a classical bias-variance trade-off in the choice of m . For all m the squared bias of the random forest is greater than that for a single tree (horizontal line).

These patterns suggest a similarity with ridge regression (Section 3.4.1). Ridge regression is useful (in linear models) when one has a large number of variables with similarly sized coefficients; ridge shrinks their coefficients toward zero, and those of strongly correlated variables toward each other. Although the size of the training sample might not permit all the variables to be in the model, this regularization via ridge stabilizes the model and allows all the variables to have their say (albeit diminished). Random forests with small m perform a similar averaging. Each of the relevant variables get their turn to be the primary split, and the ensemble averaging reduces the contribution of any individual variable. Since this simulation example (15.8) is based on a linear model in all the variables, ridge regression achieves a lower mean-squared error (about 0.45 with $\text{df}(\lambda_{\text{opt}}) \approx 29$).

15.4.3 Adaptive Nearest Neighbors

The random forest classifier has much in common with the k -nearest neighbor classifier (Section 13.3); in fact a weighted version thereof. Since each tree is grown to maximal size, for a particular Θ^* , $T(x; \Theta^*(\mathbf{Z}))$ is the response value for one of the training samples⁴. The tree-growing algorithm finds an “optimal” path to that observation, choosing the most informative predictors from those at its disposal. The averaging process assigns weights to these training responses, which ultimately vote for the prediction. Hence via the random-forest voting mechanism, those observations *close* to the target point get assigned weights—an equivalent kernel—which combine to form the classification decision.

Figure 15.11 demonstrates the similarity between the decision boundary of 3-nearest neighbors and random forests on the mixture data.

⁴We gloss over the fact that pure nodes are not split further, and hence there can be more than one observation in a terminal node

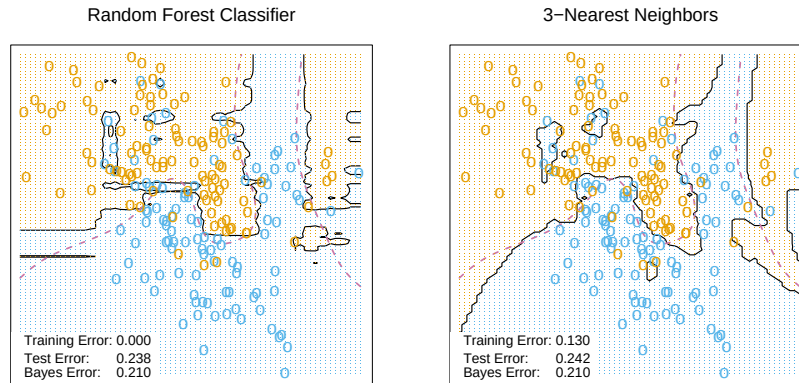


FIGURE 15.11. Random forests versus 3-NN on the mixture data. The axis-oriented nature of the individual trees in a random forest lead to decision regions with an axis-oriented flavor.

Bibliographic Notes

Random forests as described here were introduced by Breiman (2001), although many of the ideas had cropped up earlier in the literature in different forms. Notably Ho (1995) introduced the term “random forest,” and used a consensus of trees grown in random subspaces of the features. The idea of using stochastic perturbation and averaging to avoid overfitting was introduced by Kleinberg (1990), and later in Kleinberg (1996). Amit and Geman (1997) used randomized trees grown on image features for image classification problems. Breiman (1996a) introduced bagging, a precursor to his version of random forests. Dietterich (2000b) also proposed an improvement on bagging using additional randomization. His approach was to rank the top 20 candidate splits at each node, and then select from the list at random. He showed through simulations and real examples that this additional randomization improved over the performance of bagging. Friedman and Hall (2007) showed that sub-sampling (without replacement) is an effective alternative to bagging. They showed that growing and averaging trees on samples of size $N/2$ is approximately equivalent (in terms bias/variance considerations) to bagging, while using smaller fractions of N reduces the variance even further (through decorrelation).

There are several free software implementations of random forests. In this chapter we used the `randomForest` package in R, maintained by Andy Liaw, available from the CRAN website. This allows both split-variable selection, as well as sub-sampling. Adele Cutler maintains a random forest website <http://www.math.usu.edu/~adele/forests/> where (as of August 2008) the software written by Leo Breiman and Adele Cutler is freely

available. Their code, and the name “random forests”, is exclusively licensed to Salford Systems for commercial release. The **Weka** machine learning archive <http://www.cs.waikato.ac.nz/ml/weka/> at Waikato University, New Zealand, offers a free **java** implementation of random forests.

Exercises

Ex. 15.1 Derive the variance formula (15.1). This appears to fail if ρ is negative; diagnose the problem in this case.

Ex. 15.2 Show that as the number of bootstrap samples B gets large, the OOB error estimate for a random forest approaches its N -fold CV error estimate, and that in the limit, the identity is exact.

Ex. 15.3 Consider the simulation model used in Figure 15.7 (Mease and Wyner, 2008). Binary observations are generated with probabilities

$$\Pr(Y = 1|X) = q + (1 - 2q) \cdot I \left[\sum_{j=1}^J X_j > J/2 \right], \quad (15.11)$$

where $X \sim U[0, 1]^p$, $0 \leq q \leq \frac{1}{2}$, and $J \leq p$ is some predefined (even) number. Describe this probability surface, and give the Bayes error rate.

Ex. 15.4 Suppose x_i , $i = 1, \dots, N$ are iid (μ, σ^2) . Let $\bar{x}_1^*$ and $\bar{x}_2^*$ be two bootstrap realizations of the sample mean. Show that the sampling correlation $\text{corr}(\bar{x}_1^*, \bar{x}_2^*) = \frac{n}{2n-1} \approx 50\%$. Along the way, derive $\text{var}(\bar{x}_1^*)$ and the variance of the bagged mean $\bar{x}_{bag}$. Here $\bar{x}$ is a *linear* statistic; bagging produces no reduction in variance for linear statistics.

Ex. 15.5 Show that the sampling correlation between a pair of random-forest trees at a point x is given by

$$\rho(x) = \frac{\text{Var}_{\mathbf{Z}}[\mathbb{E}_{\Theta|\mathbf{Z}} T(x; \Theta(\mathbf{Z}))]}{\text{Var}_{\mathbf{Z}}[\mathbb{E}_{\Theta|\mathbf{Z}} T(x; \Theta(\mathbf{Z}))] + \mathbb{E}_{\mathbf{Z}} \text{Var}_{\Theta|\mathbf{Z}}[T(x; \Theta(\mathbf{Z}))]}. \quad (15.12)$$

The term in the numerator is $\text{Var}_{\mathbf{Z}}[\hat{f}_{\text{rf}}(x)]$, and the second term in the denominator is the expected conditional variance due to the randomization in random forests.

Ex. 15.6 Fit a series of random-forest classifiers to the **spam** data, to explore the sensitivity to the parameter m . Plot both the OOB error as well as the test error against a suitably chosen range of values for m .

Ex. 15.7 Suppose we fit a linear regression model to N observations with response y_i and predictors $x_{i1}, \dots, x_{ip}$. Assume that all variables are standardized to have mean zero and standard deviation one. Let RSS be the mean-squared residual on the training data, and $\hat{\beta}$ the estimated coefficient. Denote by RSS_j^* the mean-squared residual on the training data using the same $\hat{\beta}$, but with the N values for the j th variable randomly permuted before the predictions are calculated. Show that

$$E_P[RSS_j^* - RSS] = 2\hat{\beta}_j^2, \quad (15.13)$$

where E_P denotes expectation with respect to the permutation distribution. Argue that this is approximately true when the evaluations are done using an independent test set.

16

Ensemble Learning

16.1 Introduction

The idea of ensemble learning is to build a prediction model by combining the strengths of a collection of simpler base models. We have already seen a number of examples that fall into this category.

Bagging in Section 8.7 and random forests in Chapter 15 are ensemble methods for classification, where a *committee* of trees each cast a vote for the predicted class. Boosting in Chapter 10 was initially proposed as a committee method as well, although unlike random forests, the committee of *weak learners* evolves over time, and the members cast a weighted vote. Stacking (Section 8.8) is a novel approach to combining the strengths of a number of fitted models. In fact one could characterize any dictionary method, such as regression splines, as an ensemble method, with the basis functions serving the role of weak learners.

Bayesian methods for nonparametric regression can also be viewed as ensemble methods: a large number of candidate models are averaged with respect to the posterior distribution of their parameter settings (e.g. (Neal and Zhang, 2006)).

Ensemble learning can be broken down into two tasks: developing a population of base learners from the training data, and then combining them to form the composite predictor. In this chapter we discuss boosting technology that goes a step further; it builds an ensemble model by conducting a regularized and supervised search in a high-dimensional space of weak learners.

An early example of a learning ensemble is a method designed for multi-class classification using *error-correcting output codes* (Dietterich and Bakiri, 1995, ECOC). Consider the 10-class digit classification problem, and the coding matrix $\mathbf{C}$ given in Table 16.1.

TABLE 16.1. Part of a 15-bit error-correcting coding matrix $\mathbf{C}$ for the 10-class digit classification problem. Each column defines a two-class classification problem.

Digit	C_1	C_2	C_3	C_4	C_5	C_6	$\cdots$	C_{15}
0	1	1	0	0	0	0	$\cdots$	1
1	0	0	1	1	1	1	$\cdots$	0
2	1	0	0	1	0	0	$\cdots$	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\cdots$	$\vdots$
8	1	1	0	1	0	1	$\cdots$	1
9	0	1	1	1	0	0	$\cdots$	0

Note that the ℓ th column of the coding matrix C_ℓ defines a two-class variable that merges all the original classes into two groups. The method works as follows:

1. Learn a separate classifier for each of the $L = 15$ two class problems defined by the columns of the coding matrix.
2. At a test point x , let $\hat{p}_\ell(x)$ be the predicted probability of a one for the ℓ th response.
3. Define $\delta_k(x) = \sum_{\ell=1}^L |C_{k\ell} - \hat{p}_\ell(x)|$, the discriminant function for the k th class, where $C_{k\ell}$ is the entry for row k and column ℓ in Table 16.1.

Each row of $\mathbf{C}$ is a binary code for representing that class. The rows have more bits than is necessary, and the idea is that the redundant “error-correcting” bits allow for some inaccuracies, and can improve performance. In fact, the full code matrix $\mathbf{C}$ above has a minimum Hamming distance¹ of 7 between any pair of rows. Note that even the indicator response coding (Section 4.2) is redundant, since 10 classes require only $\lceil \log_2 10 \rceil = 4$ bits for their unique representation. Dietterich and Bakiri (1995) showed impressive improvements in performance for a variety of multiclass problems when classification trees were used as the base classifier.

James and Hastie (1998) analyzed the ECOC approach, and showed that random code assignment worked as well as the optimally constructed error-correcting codes. They also argued that the main benefit of the coding was in variance reduction (as in bagging and random forests), because the different coded problems resulted in different trees, and the decoding step (3) above has a similar effect as averaging.

¹The Hamming distance between two vectors is the number of mismatches between corresponding entries.

16.2 Boosting and Regularization Paths

In Section 10.12.2 of the first edition of this book, we suggested an analogy between the sequence of models produced by a gradient boosting algorithm and regularized model fitting in high-dimensional feature spaces. This was primarily motivated by observing the close connection between a boosted version of linear regression and the lasso (Section 3.4.2). These connections have been pursued by us and others, and here we present our current thinking in this area. We start with the original motivation, which fits more naturally in this chapter on ensemble learning.

16.2.1 Penalized Regression

Intuition for the success of the shrinkage strategy (10.41) of gradient boosting (page 364 in Chapter 10) can be obtained by drawing analogies with penalized linear regression with a large basis expansion. Consider the dictionary of all possible J -terminal node regression trees $\mathcal{T} = \{T_k\}$ that could be realized on the training data as basis functions in $\mathbb{R}^p$. The linear model is

$$f(x) = \sum_{k=1}^K \alpha_k T_k(x), \quad (16.1)$$

where $K = \text{card}(\mathcal{T})$. Suppose the coefficients are to be estimated by least squares. Since the number of such trees is likely to be much larger than even the largest training data sets, some form of regularization is required. Let $\hat{\alpha}(\lambda)$ solve

$$\min_{\alpha} \left\{ \sum_{i=1}^N \left(y_i - \sum_{k=1}^K \alpha_k T_k(x_i) \right)^2 + \lambda \cdot J(\alpha) \right\}, \quad (16.2)$$

$J(\alpha)$ is a function of the coefficients that generally penalizes larger values. Examples are

$$J(\alpha) = \sum_{k=1}^K |\alpha_k|^2 \quad \text{ridge regression}, \quad (16.3)$$

$$J(\alpha) = \sum_{k=1}^K |\alpha_k| \quad \text{lasso}, \quad (16.4)$$

$$(16.5)$$

both covered in Section 3.4. As discussed there, the solution to the lasso problem with moderate to large λ tends to be sparse; many of the $\hat{\alpha}_k(\lambda) = 0$. That is, only a small fraction of all possible trees enter the model (16.1).

Algorithm 16.1 *Forward Stagewise Linear Regression.*

1. Initialize $\check{\alpha}_k = 0$, $k = 1, \dots, K$. Set $\varepsilon > 0$ to some small constant, and M large.
2. For $m = 1$ to M :
 - (a) $(\beta^*, k^*) = \arg \min_{\beta, k} \sum_{i=1}^N \left(y_i - \sum_{l=1}^K \check{\alpha}_l T_l(x_i) - \beta T_k(x_i) \right)^2$.
 - (b) $\check{\alpha}_{k^*} \leftarrow \check{\alpha}_{k^*} + \varepsilon \cdot \text{sign}(\beta^*)$.
3. Output $f_M(x) = \sum_{k=1}^K \check{\alpha}_k T_k(x)$.

This seems reasonable since it is likely that only a small fraction of all possible trees will be relevant in approximating any particular target function. However, the relevant subset will be different for different targets. Those coefficients that are not set to zero are shrunk by the lasso in that their absolute values are smaller than their corresponding least squares values²: $|\hat{\alpha}_k(\lambda)| < |\hat{\alpha}_k(0)|$. As λ increases, the coefficients all shrink, each one ultimately becoming zero.

Owing to the very large number of basis functions T_k , directly solving (16.2) with the lasso penalty (16.4) is not possible. However, a feasible forward stagewise strategy exists that closely approximates the effect of the lasso, and is very similar to boosting and the forward stagewise Algorithm 10.2. Algorithm 16.1 gives the details. Although phrased in terms of tree basis functions T_k , the algorithm can be used with any set of basis functions. Initially all coefficients are zero in line 1; this corresponds to $\lambda = \infty$ in (16.2). At each successive step, the tree T_{k^*} is selected that best fits the current residuals in line 2(a). Its corresponding coefficient $\check{\alpha}_{k^*}$ is then incremented or decremented by an infinitesimal amount in 2(b), while all other coefficients $\check{\alpha}_k$, $k \neq k^*$ are left unchanged. In principle, this process could be iterated until either all the residuals are zero, or $\beta^* = 0$. The latter case can occur if $K < N$, and at that point the coefficient values represent a least squares solution. This corresponds to $\lambda = 0$ in (16.2).

After applying Algorithm 16.1 with $M < \infty$ iterations, many of the coefficients will be zero, namely, those that have yet to be incremented. The others will tend to have absolute values smaller than their corresponding least squares solution values, $|\check{\alpha}_k(M)| < |\hat{\alpha}_k(0)|$. Therefore this M -iteration solution qualitatively resembles the lasso, with M inversely related to λ .

Figure 16.1 shows an example, using the prostate data studied in Chapter 3. Here, instead of using trees $T_k(X)$ as basis functions, we use the origi-

²If $K > N$, there is in general no unique “least squares value,” since infinitely many solutions will exist that fit the data perfectly. We can pick the minimum L_1 -norm solution amongst these, which is the unique lasso solution.

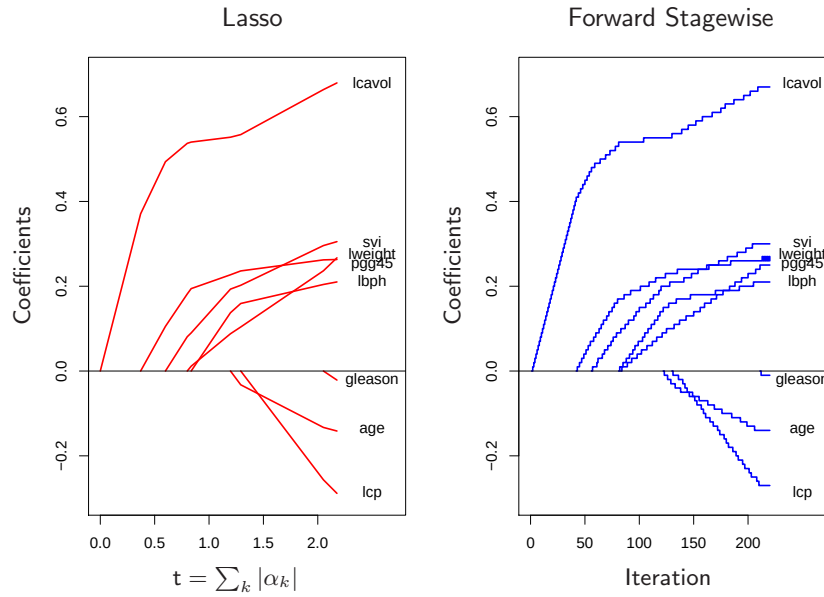


FIGURE 16.1. Profiles of estimated coefficients from linear regression, for the prostate data studied in Chapter 3. The left panel shows the results from the lasso, for different values of the bound parameter $t = \sum_k |\alpha_k|$. The right panel shows the results of the stagewise linear regression Algorithm 16.1, using $M = 220$ consecutive steps of size $\varepsilon = .01$.

nal variables X_k themselves; that is, a multiple linear regression model. The left panel displays the profiles of estimated coefficients from the lasso, for different values of the bound parameter $t = \sum_k |\alpha_k|$. The right panel shows the results of the stagewise Algorithm 16.1, with $M = 250$ and $\varepsilon = 0.01$. [The left and right panels of Figure 16.1 are the same as Figure 3.10 and the left panel of Figure 3.19, respectively.] The similarity between the two graphs is striking.

In some situations the resemblance is more than qualitative. For example, if all of the basis functions T_k are mutually uncorrelated, then as $\varepsilon \downarrow 0$, $M \uparrow$ such that $M\varepsilon \rightarrow t$, Algorithm 16.1 yields exactly the same solution as the lasso for bound parameter $t = \sum_k |\alpha_k|$ (and likewise for all solutions along the path). Of course, tree-based regressors are not uncorrelated. However, the solution sets are also identical if the coefficients $\hat{\alpha}_k(\lambda)$ are all monotone functions of λ . This is often the case when the correlation between the variables is low. When the $\hat{\alpha}_k(\lambda)$ are not monotone in λ , then the solution sets are not identical. The solution sets for Algorithm 16.1 tend to change less rapidly with changing values of the regularization parameter than those of the lasso.

Efron et al. (2004) make the connections more precise, by characterizing the exact solution paths in the ε -limiting case. They show that the coefficient paths are piece-wise linear functions, both for the lasso and forward stagewise. This facilitates efficient algorithms which allow the entire paths to be computed with the same cost as a single least-squares fit. This *least angle regression* algorithm is described in more detail in Section 3.8.1.

Hastie et al. (2007) show that this infinitesimal forward stagewise algorithm (FS₀) fits a monotone version of the lasso, which optimally reduces at each step the loss function for a given increase in the *arc length* of the coefficient path (see Sections 16.2.3 and 3.8.1). The arc-length for the $\epsilon > 0$ case is $M\epsilon$, and hence proportional to the number of steps.

Tree boosting (Algorithm 10.3) with shrinkage (10.41) closely resembles Algorithm 16.1, with the learning rate parameter ν corresponding to ε . For squared error loss, the only difference is that the optimal tree to be selected at each iteration T_{k*} is approximated by the standard top-down greedy tree-induction algorithm. For other loss functions, such as the exponential loss of AdaBoost and the binomial deviance, Rosset et al. (2004a) show similar results to what we see here. Thus, one can view tree boosting with shrinkage as a form of monotone ill-posed regression on all possible (J -terminal node) trees, with the lasso penalty (16.4) as a regularizer. We return to this topic in Section 16.2.3.

The choice of no shrinkage [$\nu = 1$ in equation (10.41)] is analogous to forward-stepwise regression, and its more aggressive cousin best-subset selection, which penalizes the *number* of non zero coefficients $J(\alpha) = \sum_k |\alpha_k|^0$. With a small fraction of dominant variables, best subset approaches often work well. But with a moderate fraction of strong variables, it is well known that subset selection can be excessively greedy (Copas, 1983), often yielding poor results when compared to less aggressive strategies such as the lasso or ridge regression. The dramatic improvements often seen when shrinkage is used with boosting are yet another confirmation of this approach.

16.2.2 The “Bet on Sparsity” Principle

As shown in the previous section, boosting’s forward stagewise strategy with shrinkage approximately minimizes the same loss function with a lasso-style L_1 penalty. The model is built up slowly, searching through “model space” and adding shrunken basis functions derived from important predictors. In contrast, the L_2 penalty is computationally much easier to deal with, as shown in Section 12.3.7. With the basis functions and L_2 penalty chosen to match a particular positive-definite kernel, one can solve the corresponding optimization problem without explicitly searching over individual basis functions.

However, the sometimes superior performance of boosting over procedures such as the support vector machine may be largely due to the implicit use of the L_1 versus L_2 penalty. The shrinkage resulting from the

L_1 penalty is better suited to *sparse* situations, where there are few basis functions with nonzero coefficients (among all possible choices).

We can strengthen this argument through a simple example, taken from Friedman et al. (2004). Suppose we have 10,000 data points and our model is a linear combination of a million trees. If the true population coefficients of these trees arose from a Gaussian distribution, then we know that in a Bayesian sense the best predictor is ridge regression (Exercise 3.6). That is, we should use an L_2 rather than an L_1 penalty when fitting the coefficients. On the other hand, if there are only a small number (e.g., 1000) coefficients that are nonzero, the lasso (L_1 penalty) will work better. We think of this as a *sparse* scenario, while the first case (Gaussian coefficients) is *dense*. Note however that in the dense scenario, although the L_2 penalty is best, neither method does very well since there is too little data from which to estimate such a large number of nonzero coefficients. This is the curse of dimensionality taking its toll. In a sparse setting, we can potentially do well with the L_1 penalty, since the number of nonzero coefficients is small. The L_2 penalty fails again.

In other words, use of the L_1 penalty follows what we call the “bet on sparsity” principle for high-dimensional problems:

Use a procedure that does well in sparse problems, since no procedure does well in dense problems.

These comments need some qualification:

- For any given application, the degree of sparseness/denseness depends on the unknown true target function, and the chosen dictionary $\mathcal{T}$.
- The notion of sparse versus dense is relative to the size of the training data set and/or the noise-to-signal ratio (NSR). Larger training sets allow us to estimate coefficients with smaller standard errors. Likewise in situations with small NSR, we can identify more nonzero coefficients with a given sample size than in situations where the NSR is larger.
- The size of the dictionary plays a role as well. Increasing the size of the dictionary may lead to a sparser representation for our function, but the search problem becomes more difficult leading to higher variance.

Figure 16.2 illustrates these points in the context of linear models using simulation. We compare ridge regression and lasso, both for classification and regression problems. Each run has 50 observations with 300 independent Gaussian predictors. In the top row all 300 coefficients are nonzero, generated from a Gaussian distribution. In the middle row, only 10 are nonzero and generated from a Gaussian, and the last row has 30 non zero Gaussian coefficients. For regression, standard Gaussian noise is

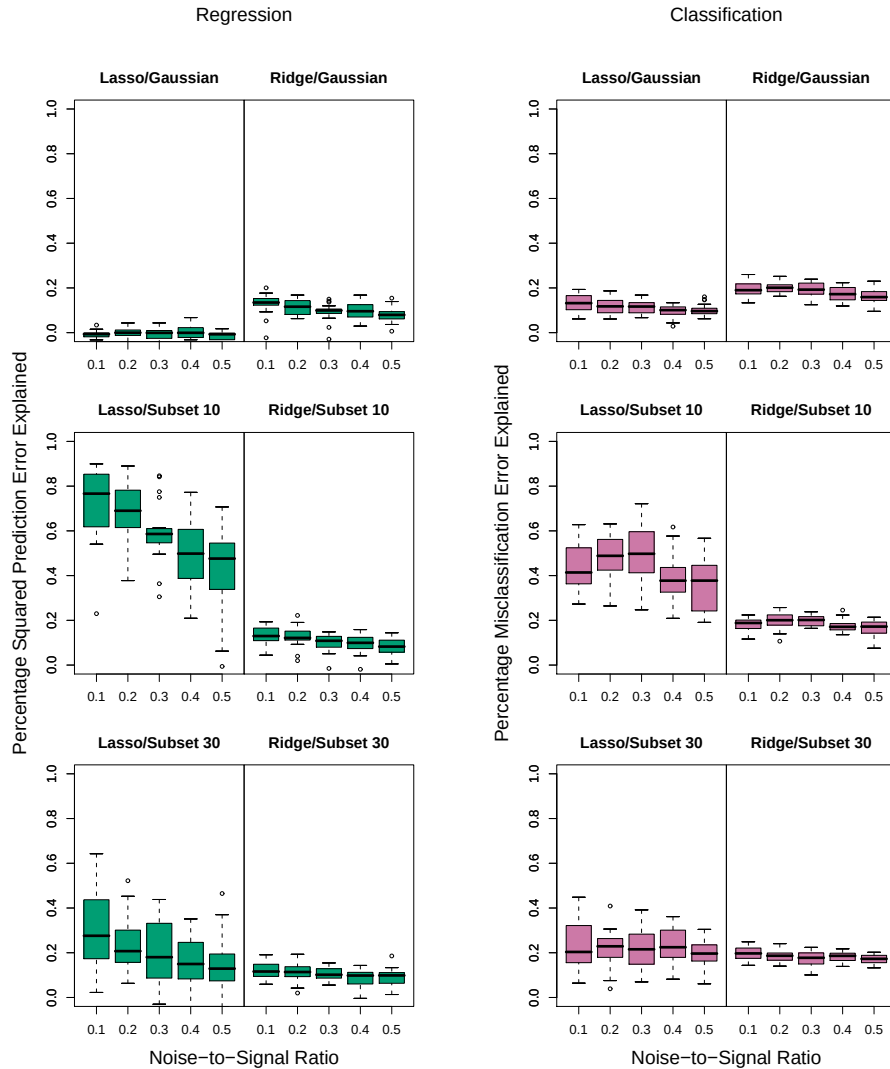


FIGURE 16.2. Simulations that show the superiority of the L_1 (lasso) penalty over L_2 (ridge) in regression and classification. Each run has 50 observations with 300 independent Gaussian predictors. In the top row all 300 coefficients are nonzero, generated from a Gaussian distribution. In the middle row, only 10 are nonzero, and the last row has 30 nonzero. Gaussian errors are added to the linear predictor $\eta(X)$ for the regression problems, and binary responses generated via the inverse-logit transform for the classification problems. Scaling of $\eta(X)$ resulted in the noise-to-signal ratios shown. Lasso is used in the left sub-columns, ridge in the right. We report the optimal percentage of error explained on test data (relative to the error of a constant model), displayed as boxplots over 20 realizations for each combination. In the only situation where ridge beats lasso (top row), neither do well.

added to the linear predictor $\eta(X) = X^T \beta$ to produce a continuous response. For classification the linear predictor is transformed via the inverse-logit to a probability, and a binary response is generated. Five different noise-to-signal ratios are presented, obtained by scaling $\eta(X)$ prior to generating the response. In both cases this is defined to be $NSR = \text{Var}(Y|\eta(X))/\text{Var}(\eta(X))$. Both the ridge regression and lasso coefficient paths were fit using a series of 50 values of λ corresponding to a range of df from 1 to 50 (see Chapter 3 for details). The models were evaluated on a large test set (infinite for Gaussian, 5000 for binary), and in each case the value for λ was chosen to minimize the test-set error. We report percentage variance explained for the regression problems, and percentage misclassification error explained for the classification problems (relative to a baseline error of 0.5). There are 20 simulation runs for each scenario.

Note that for the classification problems, we are using squared-error loss to fit the binary response. Note also that we do not use the training data to select λ , but rather are reporting the best possible behavior for each method in the different scenarios. The L_2 penalty performs poorly everywhere. The Lasso performs reasonably well in the only two situations where it can (sparse coefficients). As expected the performance gets worse as the NSR increases (less so for classification), and as the model becomes denser. The differences are less marked for classification than for regression.

These empirical results are supported by a large body of theoretical results (Donoho and Johnstone, 1994; Donoho and Elad, 2003; Donoho, 2006b; Candes and Tao, 2007) that support the superiority of L_1 estimation in sparse settings.

16.2.3 Regularization Paths, Over-fitting and Margins



It has often been observed that boosting “does not overfit,” or more accurately is “slow to overfit.” Part of the explanation for this phenomenon was made earlier for random forests — misclassification error is less sensitive to variance than is mean-squared error, and classification is the major focus in the boosting community. In this section we show that the regularization paths of boosted models are “well behaved,” and that for certain loss functions they have an appealing limiting form.

Figure 16.3 shows the coefficient paths for lasso and infinitesimal forward stagewise (FS_0) in a simulated regression setting. The data consists of a dictionary of 1000 Gaussian variables, strongly correlated ($\rho = 0.95$) within blocks of 20, but uncorrelated between blocks. The generating model has nonzero coefficients for 50 variables, one drawn from each block, and the coefficient values are drawn from a standard Gaussian. Finally, Gaussian noise is added, with a noise-to-signal ratio of 0.72 (Exercise 16.1.) The FS_0 algorithm is a limiting form of algorithm 16.1, where the step size ε is shrunk to zero (Section 3.8.1). The grouping of the variables is intended to mimic the correlations of nearby trees, and with the forward-stagewise

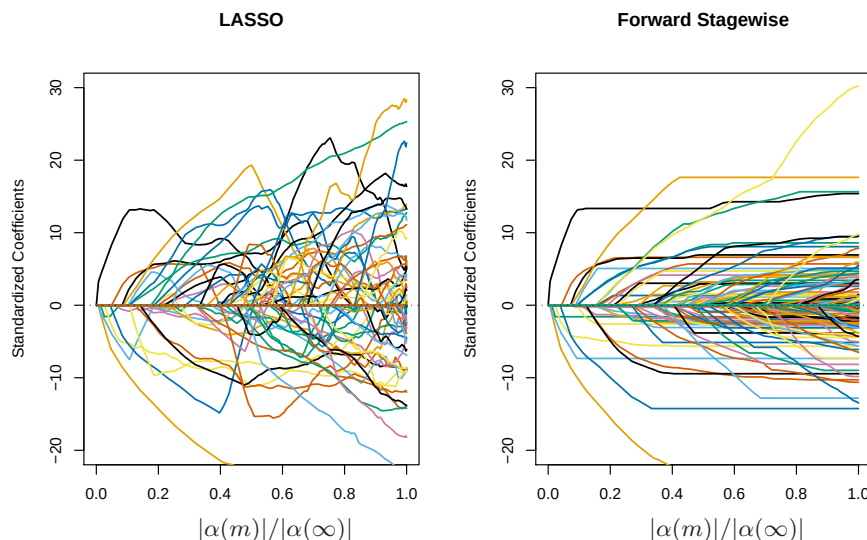


FIGURE 16.3. Comparison of lasso and infinitesimal forward stagewise paths on simulated regression data. The number of samples is 60 and the number of variables is 1000. The forward-stagewise paths fluctuate less than those of lasso in the final stages of the algorithms.

algorithm, this setup is intended as an idealized version of gradient boosting with shrinkage. For both these algorithms, the coefficient paths can be computed exactly, since they are piecewise linear (see the LARS algorithm in Section 3.8.1).

Here the coefficient profiles are similar only in the early stages of the paths. For the later stages, the forward stagewise paths tend to be monotone and smoother, while those for the lasso fluctuate widely. This is due to the strong correlations among subsets of the variables —lasso suffers somewhat from the multi-collinearity problem (Exercise 3.28).

The performance of the two models is rather similar (Figure 16.4), and they achieve about the same minimum. In the later stages forward stagewise takes longer to overfit, a likely consequence of the smoother paths.

Hastie et al. (2007) show that FS_0 solves a *monotone* version of the lasso problem for squared error loss. Let $\mathcal{T}^a = \mathcal{T} \cup \{-\mathcal{T}\}$ be the augmented dictionary obtained by including a negative copy of every basis element in $\mathcal{T}$. We consider models $f(x) = \sum_{T_k \in \mathcal{T}^a} \alpha_k T_k(x)$ with non-negative coefficients $\alpha_k \geq 0$. In this expanded space, the lasso coefficient paths are positive, while those of FS_0 are monotone nondecreasing.

The monotone lasso path is characterized by a differential equation

$$\frac{\partial \alpha}{\partial \ell} = \rho^{ml}(\alpha(\ell)), \quad (16.6)$$

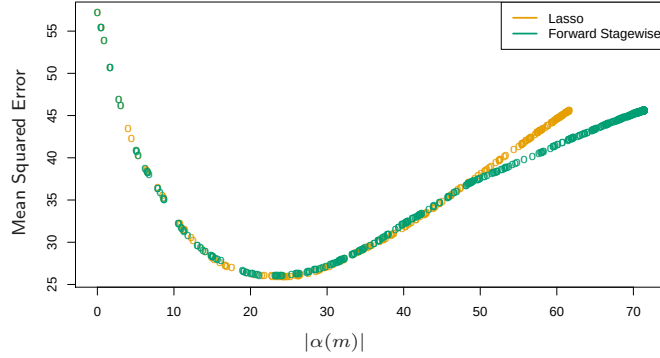


FIGURE 16.4. Mean squared error for lasso and infinitesimal forward stagewise on the simulated data. Despite the difference in the coefficient paths, the two models perform similarly over the critical part of the regularization path. In the right tail, lasso appears to overfit more rapidly.

with initial condition $\alpha(0) = 0$, where ℓ is the L_1 arc-length of the path $\alpha(\ell)$ (Exercise 16.2). The monotone lasso move direction (velocity vector) $\rho^{ml}(\alpha(\ell))$ decreases the loss at the optimal quadratic rate per unit increase in the L_1 arc-length of the path. Since $\rho_k^{ml}(\alpha(\ell)) \geq 0 \forall k, \ell$, the solution paths are monotone.

The lasso can similarly be characterized as the solution to a differential equation as in (16.6), except that the move directions decrease the loss optimally per unit increase in the L_1 norm of the path. As a consequence, they are not necessarily positive, and hence the lasso paths need not be monotone.

In this augmented dictionary, restricting the coefficients to be positive is natural, since it avoids an obvious ambiguity. It also ties in more naturally with tree boosting—we always find trees positively correlated with the current residual.

There have been suggestions that boosting performs well (for two-class classification) because it exhibits maximal-margin properties, much like the support-vector machines of Chapters 4.5.2 and 12. Schapire et al. (1998) define the *normalized L_1 margin* of a fitted model $f(x) = \sum_k \alpha_k T_k(x)$ as

$$m(f) = \min_i \frac{y_i f(x_i)}{\sum_{k=1}^K |\alpha_k|}. \quad (16.7)$$

Here the minimum is taken over the training sample, and $y_i \in \{-1, +1\}$. Unlike the L_2 margin (4.40) of support vector machines, the L_1 margin $m(f)$ measures the distance to the closest training point in L_∞ units (maximum coordinate distance).

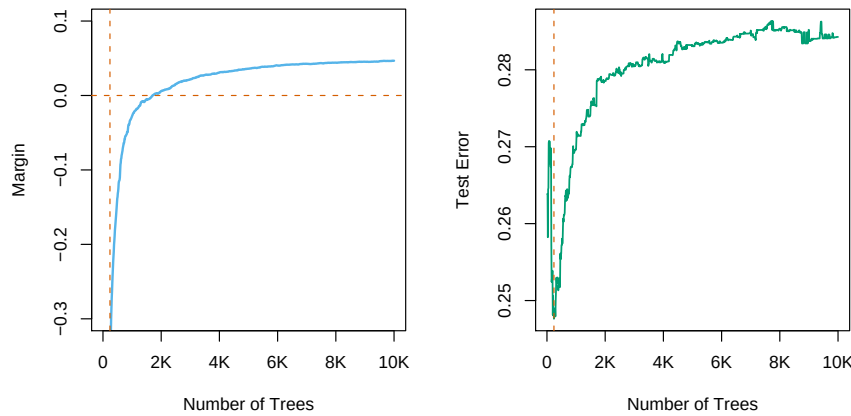


FIGURE 16.5. The left panel shows the L_1 margin $m(f)$ for the Adaboost classifier on the mixture data, as a function of the number of 4-node trees. The model was fit using the R package `gbm`, with a shrinkage factor of 0.02. After 10,000 trees, $m(f)$ has settled down. Note that when the margin crosses zero, the training error becomes zero. The right panel shows the test error, which is minimized at 240 trees. In this case, Adaboost overfits dramatically if run to convergence.

Schapire et al. (1998) prove that with separable data, Adaboost increases $m(f)$ with each iteration, converging to a margin-symmetric solution. Rätsch and Warmuth (2002) prove the asymptotic convergence of Adaboost with shrinkage to a L_1 -margin-maximizing solution. Rosset et al. (2004a) consider regularized models of the form (16.2) for general loss functions. They show that as $\lambda \downarrow 0$, for particular loss functions the solution converges to a margin-maximizing configuration. In particular they show this to be the case for the exponential loss of Adaboost, as well as binomial deviance.

Collecting together the results of this section, we reach the following summary for boosted classifiers:

The sequence of boosted classifiers form an L_1 -regularized monotone path to a margin-maximizing solution.

Of course the margin-maximizing end of the path can be a very poor, overfit solution, as it is in the example in Figure 16.5. Early stopping amounts to picking a point along the path, and should be done with the aid of a validation dataset.

16.3 Learning Ensembles

The insights learned from the previous sections can be harnessed to produce a more effective and efficient ensemble model. Again we consider functions

of the form

$$f(x) = \alpha_0 + \sum_{T_k \in \mathcal{T}} \alpha_k T_k(x), \quad (16.8)$$

where $\mathcal{T}$ is a dictionary of basis functions, typically trees. For gradient boosting and random forests, $|\mathcal{T}|$ is very large, and it is quite typical for the final model to involve many thousands of trees. In the previous section we argue that gradient boosting with shrinkage fits an L_1 regularized monotone path in this space of trees.

Friedman and Popescu (2003) propose a hybrid approach which breaks this process down into two stages:

- A finite dictionary $\mathcal{T}_L = \{T_1(x), T_2(x), \dots, T_M(x)\}$ of basis functions is induced from the training data;
- A family of functions $f_\lambda(x)$ is built by fitting a lasso path in this dictionary:

$$\alpha(\lambda) = \arg \min_{\alpha} \sum_{i=1}^N L[y_i, \alpha_0 + \sum_{m=1}^M \alpha_m T_m(x_i)] + \lambda \sum_{m=1}^M |\alpha_m|. \quad (16.9)$$

In its simplest form this model could be seen as a way of post-processing boosting or random forests, taking for $\mathcal{T}_L$ the collection of trees produced by the gradient boosting or random forest algorithms. By fitting the lasso path to these trees, we would typically use a much reduced set, which would save in computations and storage for future predictions. In the next section we describe modifications of this prescription that reduce the correlations in the ensemble $\mathcal{T}_L$, and improve the performance of the lasso post processor.

As an initial illustration, we apply this procedure to a random forest ensemble grown on the spam data.

Figure 16.6 shows that a lasso post-processing offers modest improvement over the random forest (blue curve), and reduces the forest to about 40 trees, rather than the original 1000. The post-processed performance matches that of gradient boosting. The orange curves represent a modified version of random forests, designed to reduce the correlations between trees even more. Here a random sub-sample (without replacement) of 5% of the training sample is used to grow each tree, and the trees are restricted to be shallow (about six terminal nodes). The post-processing offers more dramatic improvements here, and the training costs are reduced by a factor of about 100. However, the performance of the post-processed model falls somewhat short of the blue curves.

16.3.1 Learning a Good Ensemble

Not all ensembles $\mathcal{T}_L$ will perform well with post-processing. In terms of basis functions, we want a collection that covers the space well in places